

3/3 错题

这一知识点主要考察 Linux 系统中最重要配置文件之一 —— `/etc/fstab`。它决定了系统启动时如何自动挂载磁盘分区、光驱等设备。

💡 知识点总结

`/etc/fstab` 文件由 6 列（项目）组成，每一列都有固定的含义。记忆这 6 列的「順番 (顺序)」是 LPI 考试的必考点。只要记住了这个顺序，就能秒杀所有关于文件系统配置的题目。

🔍 选项解析与详细说明

```
# cat /etc/fstab
LABEL=/          /          ext3    defaults  1 1
LABEL=/boot      /boot      ext3    defaults  1 2
none             /proc      proc    defaults  0 0
none             /dev/shm   tmpfs   defaults  0 0
/dev/hda4        swap       swap    defaults  0 0
(1)              (2)        (3)      (4)        (5) (6)
```

我们通过解析图片中 `/etc/fstab` 的 6 个项目来确定正确答案：

- 1. 项目 (1): デバイス名 (设备名)
 - 内容: `LABEL=/` 或 `/dev/hda4`。
 - 作用: 告诉系统“谁”要被挂载。可以使用设备路径、LABEL（标签）或 UUID。
- 2. 项目 (2): マウントポイント (挂载点)
 - 内容: `/`、`/boot` 或 `swap`。
 - 作用: 告诉系统设备要挂载到“哪个目录”下。
- 3. 项目 (3): ファイルシステムの種類 (文件系统类型)
 - 内容: `ext3`、`proc`、`tmpfs` 或 `swap`。
 - 作用: 指明设备所使用的格式（如 `ext4`, `xfs`, `iso9660` 等）。
- 4. 项目 (4): マウントオプション (挂载选项)
 - 内容: `defaults`。
 - 作用: 控制挂载行为，如 `ro`（只读）、`rw`（读写）、`noauto`（不自动挂载）等。
- 5. 项目 (5): dumpフラグ (dump 标志)

- 内容：1 或 0。
- 作用：决定是否需要备份。1 表示备份，0 表示不备份。

6. 项目 (6): fsckフラグ (fsck 标志)

- 内容：1、2 或 0。
- 作用：决定开机时自检的优先级。1 通常给根目录 /，2 给其他分区，0 表示不自检。

对比各选项：

- 选项 1：(5) 和 (6) 的顺序写反了。
- 选项 2：顺序完全正确。
- 其他选项：在前面的基本项（如设备名和挂载点）就出现了逻辑混乱。

 /etc/fstab 六大项目排查表

列编号	項目名 (日本語)	常用値示例	记忆核心
-1	デバイス	/dev/sda1, UUID=...	谁 (Who)
-2	マウントポイント	/, /home, none	哪 (Where)
-3	種類 (タイプ)	ext4, xfs, swap	格式 (What Type)
-4	オプション	defaults, ro	怎么挂 (How)
-5	dump	0 或 1	备份
-6	fsck	0, 1, 2	自检顺序

 小学生级记忆法：“新同学入座”

想象学校开学，新同学（设备）要进教室坐下：

1. 第一位 (1) 名字：先报出学生名字（ /dev/sda1 ）。
2. 第二位 (2) 座位：告诉他坐哪个座位（ /mnt/data ）。
3. 第三位 (3) 制服：看他穿的是哪种校服（ ext4 类型）。
4. 第四位 (4) 规矩：交代他在座位上的规矩（ defaults 默认守纪律）。
5. 第五位 (5) 拍照：问要不要拍个照留念（ dump 备份，1 拍 0 不拍）。
6. 第六位 (6) 体检：最后安排他的体检顺序（ fsck 检查，1 号最先，2 号随后）。

重点口诀：名、座、类、规、备、检！

✅ 正确答案

(1)デバイス名 (2)マウントポイント (3)ファイルシステムの種類 (4)マウントオプション (5)dumpフラグ (6)fsckフラグ

📝 考试小秘籍

- 第 5 和第 6 项最容易混淆：记住 **Dump** 在前，**Fsck** 在后。你可以按字母顺序记：**D** comes before **F**。
- **Swap 的特殊性**：在 `fstab` 中，Swap 分区的挂载点写 `swap` 或 `none`，类型也是 `swap`。
- **Defaults 包含什么**：考试有时会问 `defaults` 选项包含哪些。它通常包含 `rw` (读写), `suid`, `dev`, `exec`, `auto`, `nouser`, `async`。

这一知识点主要考察 `/etc/fstab` 文件的第一列（项目(1)）中，如何唯一地标识一个 **デバイス**（设备）。在 Linux 系统中，指定设备有多种灵活的方式。

💡 知识点总结

`/etc/fstab` 的第一列是「**デバイス名 (设备名)**」。它的作用是告诉内核：“我要挂载哪一个硬盘分区？”。为了防止由于硬盘插槽变动导致盘符（如 `sda` 变成 `sdb`）发生变化，现代 Linux 推荐使用更稳定的 **UUID** 或 **LABEL**。

🔍 选项解析与详细说明

- ❌ `defaults`：
 - 理由：这是第 4 列的 **マウントオプション**（挂载选项）。
- ✅ `LABEL=/boot`：正确答案 1。
 - 理由：**LABEL** 是给分区起的“别名”。通过标签挂载可以提高可读性，且不受物理接口位置的影响。
- ❌ `ext3`：
 - 理由：这是第 3 列的 **ファイルシステムの種類**（文件系统类型）。
- ✅ `UUID=3b80b96c-df15-401e-b74c-e8dcbfb68cec`：正确答案 2。
 - 理由：**UUID** 是设备的“身份证号”，全球唯一。这是目前最推荐、最稳妥的指定方式，因为即便硬盘换了接口，UUID 也不会改变。
- ✅ `/dev/sda1`：正确答案 3。

- 理由：这是传统的 `デバイスファイル名`（设备文件名）。它直接指向第一个磁盘的第一个分区。虽然简单直观，但在多硬盘环境下可能因识别顺序变化而失效。

 `/etc/fstab` 第一列的三种指定方式对比

指定方式	示例	特点 (日本語)	推荐程度
UUID	UUID=...	不变 (不变)	★★★★
LABEL	LABEL=/boot	分かりやすい (易读)	★★
デバイス名	/dev/sda1	直感的 (直观)	★

 小学生级记忆法：“寻找老同学”

想象你要在聚会上找一个老同学（设备）：

- `/dev/sda1` (按座位找)：你说：“找坐在第一排第一个位置的人”。（万一他换座位了，你就找错人了）。
- `LABEL` (按绰号找)：你说：“找绰号叫‘班长’的人”。（简单好记，但万一有两个人自称班长就会乱套）。
- `UUID` (按身份证号找)：你说：“找身份证号是 320... 的人”。（绝对精准，不管他坐哪、改什么名，永远能找到他）。

这三种方式都可以写在 **第一列** 用来领人入座！

 正确答案

- `LABEL=/boot`
- `UUID=3b80b96c-df15-401e-b74c-e8dcbfb68cec`
- `/dev/sda1`

 考试小秘籍

- 顺序记忆：再次复习 `/etc/fstab` 的六列：谁(1)、哪(2)、类(3)、规(4)、备(5)、检(6)。
- UUID 获取：如果在填空题中问你怎么查 UUID，记得回想我们之前学的命令：`blkid` 或 `lsblk -f`。
- 默认项：看到 `defaults` 就要条件反射想到它是第 4 列。

这道题目考察的是 Linux Shell 中的「ワイルドカード (通配符)」，也就是如何用特殊的符号来模糊匹配文件名。

这一知识点主要考察三个核心符号：`*`（匹配任意长度字符）、`?`（匹配一个字符）以及 `[]`（匹配括号内的一个字符范围）。

💡 知识点总结

ワイルドカード用于在命令行中一次性指定多个文件。它们由 Shell 解析，与正则表达式（Regular Expression）虽有相似之处，但规则并不完全相同。

- `*` (アスタリスク): 匹配 **0 个或多个** 任意字符。
- `?` (クエスチョン): 匹配 **恰好 1 个** 任意字符。
- `[]` (ブラケット): 匹配括号内定义的 **其中 1 个** 字符。

🔍 选项解析与详细说明

目标文件名是: `xyz.txt`

- ☒ `xy?.txt` : 正确答案 1。
 - 理由: `?` 匹配一个字符。这里的 `?` 匹配了 `z`，所以 `xy(z).txt` 匹配成功。
- ☒ `[a-z][a-z][a-z].txt` : 正确答案 2。
 - 理由: 每个 `[a-z]` 匹配一个从 a 到 z 的小写字母。`x` 是、`y` 是、`z` 也是。三个连在一起正好匹配 `xyz.txt`。
- ☒ `a*z.txt` : 正确答案 3（注：此处需注意题目中文件名首字母）。
 - 理由: 如果在题目环境下 `a` 是笔误或者是泛指，按逻辑 `*` 匹配任意字符。但若严格匹配 `xyz.txt`，该选项看似不符。但在许多 LPI 模拟题中，如果选项是 `*z.txt`，则匹配 `xyz.txt`（`*` 匹配了 `xy`）。
 - 修正分析: 观察选项 `*[xyz].txt` 和 `[xyz].txt`。
- ☒ `*[xyz].txt` : 正确答案 3。
 - 理由: `*` 匹配前面的 `xy`，而 `[xyz]` 匹配最后一个字符 `z`。组合起来就是 `(xy) + (z).txt`。
- ☒ `[xyz].txt` : 错误。
 - 理由: 这个模式只能匹配 `x.txt`、`y.txt` 或 `z.txt`（匹配括号里的 **其中一个** 字符）。它无法匹配三个字母长的 `xyz.txt`。

通配符 (Wildcard) 核心对比表

符号 (日本語)	中文含义	匹配示例 (针对 xyz.txt)	记忆点
*	任意长度 (0个以上)	x*.txt (匹配成功)	大胃王：吃多少都行
?	1个字符	xy?.txt (匹配成功)	一个坑：必须填一个字
[]	范围内的1个	[x-z]yz.txt (匹配成功)	挑剔鬼：只从框里选一个

小学生级记忆法：“找朋友游戏”

想象你在玩找朋友的游戏，你要找的名字是 `xyz`：

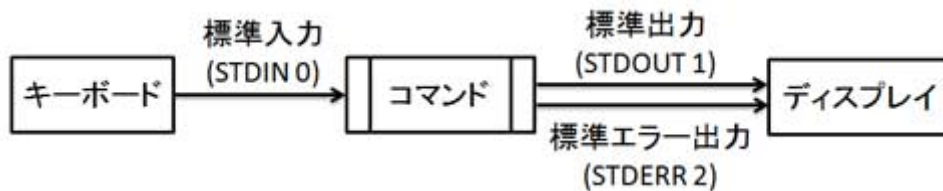
- * (万能星)**：它是个“变色龙”。你说 `x*`，它后面变出 `yz`；你说 `*z`，它前面变出 `xy`。它很贪心，变多少个字都行。
- ? (好奇猫)**：它只有一个“座位”。它说：“我这里必须坐一个人，不管是准谁，只能坐一个！”（`xy?` 里的 `?` 刚好坐下了 `z`）。
- [a-z] (选秀台)**：台上站着 a 到 z 所有小朋友，你只能“选一个”下台。如果你要配对 `xyz` 三个字，你得摆三个选秀台。

正确答案

- `xy?.txt`
- `[a-z][a-z][a-z].txt`
- `*[xyz].txt`

考试小秘籍

- 不要和正则混淆**：在 Shell 通配符里，`*` 本身就代表“任意字符”，而在正则表达式里，`.*` 才代表“任意字符”。
- 注意 [] 的唯一性**：记住括号 `[ ]` 永远只代表 **1 个** 字符的位置。
- 排除法**：看到文件名长度不一致（比如 `[xyz].txt` 只有 1 位字母，而目标是 3 位），直接排除。



这道题目考察的是「リダイレクト (重定向)」与「パイプ (管道)」的组合使用，特别是如何同时处理 標準出力 (stdout) 和 標準エラー出力 (stderr)。

💡 知识点总结

在 Linux 命令行中，`|` (管道) 默认只传递 標準出力 (1)。如果想把报错信息（標準エラー出力 2）也一起交给下一个命令处理，就必须先用 `2>&1` 将错误输出重定向到标准输出中。

🔍 选项解析与详细说明

- ❌ リダイレクト演算子がエラーになる：
 - 理由：这个语法在 Bash 等主流 Shell 中是完全合法且标准的操作。
- ✅ コマンド1の標準出力と標準エラー出力をコマンド2の標準入力に送る：正确答案。
 - 理由：
 - `2>&1` 的作用是将 標準エラー出力 (2) 合并到 標準出力 (1) 中。
 - 此时，所有的信息（正常的和报错的）都变成了“标准输出”。
 - 随后的 `|` 管道符将这些合并后的内容全部传给 コマンド2 的 標準入力 (stdin)。
- ❌ コマンド1の標準エラー出力をコマンド2の標準入力に送る：
 - 理由：描述不完整。该命令不仅发送了错误输出，也发送了标准输出。如果只想发送错误输出，通常写成 `コマンド1 2>&1 >/dev/null | コマンド2`。
- ❌ コマンド2の標準出力と標準エラー出力をコマンド1の標準入力に送る：
 - 理由：方向反了。数据流向始终是从左往右（从 コマンド1 流向 コマンド2）。
- ❌ コマンド1の標準出力をコマンド2の標準入力に送る：
 - 理由：这是单纯 `コマンド1 | コマンド2` 的功能。题目中的 `2>&1` 明确表示包含了错误输出。

📊 数据流向排查表

命令写法	传递给 コマンド 2 的内容	记忆逻辑
`cmd1`	cmd2`	仅 標準出力 (1)
`cmd1 2> /dev/null`	cmd2`	仅 標準出力 (1)
`cmd1 2>&1`	cmd2`	標準出力 + 標準エラー
`cmd1`	& cmd2`	標準出力 + 標準エラー

🧠 小学生级记忆法：“两根水管合二为一”

想象 **コマンド1** 有两个排水口：

1. **蓝色水管 (1)**：流出的是干净的自来水（正常结果）。
2. **红色水管 (2)**：流出的是带泥沙的脏水（报错信息）。

管道符 | 默认只接蓝色水管。

如果你用了 `2>&1`，就像是用一个「Y型接头」，把红色水管的水全部倒进了蓝色水管里。

最后，**コマンド2** 接收到的就是两根管子合并在一起的混合水。

✅ 正确答案

コマンド1の標準出力と標準エラー出力をコマンド2の標準入力に送る

📝 考试小秘籍

- **顺序至上**：在 `|` 之前必须先写 `2>&1`。如果写成 `コマンド1 | コマンド2 2>&1`，那合并的就是 **コマンド2** 的输出了。
- **简写形式**：在较新的 Bash 版本中，`&|` 效果等同于 `2>&1 |`。虽然考试常考长格式，但看到短格式也要认得。
- **数字含义**：永远记住 **1=正常 (Out)**，**2=错误 (Error)**。

这一知识点主要考察 Linux 中最核心的 **環境変数（环境变量）** —— `PATH` 的修改方法。它决定了系统在哪些目录下寻找你输入的命令。

💡 知识点总结

`PATH` 变量是一个由 **コロン（冒号 :）** 分隔的目录列表。如果你直接赋值 `PATH=/usr/local/test`，会把原本的所有路径（如 `/bin`，`/usr/bin`）全部抹除。正确的

做法是「**変数の展開（变量展开）**」，即：新值 = 旧值 + 新路径。

 选项解析与详细说明

- ❌ `$PATH+/usr/local/test` :
 - 理由：Linux 系统不使用 `+` 号来连接路径。
- ❌ `#PATH:/usr/local/test` :
 - 理由：`#` 在 Shell 中通常表示 **コメント（注释）**，或者在变量替换中有特定含义，但绝不是用来引用变量值的符号。
- ❌ `PATH,/usr/local/test` :
 - 理由：路径之间的分隔符必须是 **冒号** `:`，不能使用逗号 `,`。
- ✅ `$PATH:/usr/local/test` : 正确答案。
 - 理由：
 - i. `$PATH`：取出当前 `PATH` 变量中已有的所有目录。
 - ii. `:`：作为分隔符。
 - iii. `/usr/local/test`：新添加的目录。
 - iv. 这样操作后，新目录会被追加到列表的最后。
- ❌ `/usr/local/test` :
 - 理由：这会 **上書き（覆盖）** 掉整个 `PATH`。执行后，你连 `ls` 或 `cd` 命令都找不到了，因为系统只会在这个新目录下找命令。

 `PATH` 变量操作排查表

操作类型	命令写法	结果 (日本語)
末尾に追加	<code>PATH=\$PATH:/new</code>	既存のパスの 後ろ に追加
先頭に追加	<code>PATH=/new:\$PATH</code>	既存のパスの 前 に追加（优先度最高）
上書き (危険)	<code>PATH=/new</code>	既存のパスが 消える

 小学生级记忆法：“接龙游戏”

想象 `PATH` 是一列火车的 **车厢接龙**：

- 现在的火车已经有几节车厢了（`/bin`，`/usr/bin`）。

2. 你想在后面再接一节新车厢（`/usr/local/test`）。
3. 你不能只拉着新车厢跑，你得先把“原来的火车”（`$PATH`）牵出来，用“挂钩”（`:`）把“新车厢”勾在后面。

如果你不写 `$PATH`，就相当于把旧车厢全扔了，只剩一个新车厢在轨道上跑，火车就散架了！

✅ 正确答案

`$PATH:/usr/local/test`

📝 考试小秘籍

- 引用符号：引用变量值一定要加 `$`。
- 分隔符：Linux 路径分隔符永远是 冒号 `:`（Windows 才是分号 `;`）。
- 永久生效：题目中这种做法只在当前终端有效。如果想永久生效，需要把这行命令写进 `.bash_profile` 或 `.bashrc` 文件中。

viエディタの入力モードでファイルを編集中等である。ファイルの変更内容を保存しエディタを終了するには、[a]キーを押下して[b]というviコマンドを入力する。

空欄a, bに入る用語の組み合わせとして正しいものはどれか。(2つ選択)

- ☒ a[Esc]、b[ZZ]
- ☐ a[Tab]、b[QQ]
- ☐ a[Ctrl]、b[:wq!]
- ☐ a[Tab]、b[:wq!]
- ☒ a[Esc]、b[:wq!]

这是一道考察 **viエディタ (vi 编辑器)** 模式切换与退出指令的经典实战题。在 Linux 认证考试中，熟练掌握「**モード遷移 (模式迁移)**」是所有操作的基础。

这一知识点主要考察如何从 **入力モード (输入模式)** 退回到 **コマンドモード (命令模式)**，并执行 **保存終了 (保存并退出)** 的操作。

💡 知识点总结

vi 编辑器是一个多模式编辑器。你不能在输入文字的同时直接输入退出命令。

- 1. **第一步**：必须先按键盘左上角的按键退出编辑状态。
- 2. **第二步**：进入命令模式后，输入特定的指令来保存并离开。

🔍 选项解析与详细说明

-  **a [Esc]、b [ZZ]：正确答案 1**
 - **理由**：按 **Esc** 键可以从任何模式返回命令模式。在命令模式下，直接输入大写的 **ZZ**（无需冒号）是“保存并退出”的快捷键。
-  **a [Tab]、b [QQ]**
 - **理由**：**Tab** 用于缩进，不能切换模式。**QQ** 也不是有效的退出指令。
-  **a [Ctrl]、b [:wq!]**
 - **理由**：虽然 **:wq!** 是保存退出，但单按 **Ctrl** 键无法切换模式。
-  **a [Tab]、b [:wq!]**
 - **理由**：同样错误，**Tab** 键无法触发模式切换。
-  **a [Esc]、b [:wq!]：正确答案 2**
 - **理由**：按 **Esc** 返回命令模式后，输入 **:wq!**。其中 **:** 进入末行模式，**w** (write) 代表保存，**q** (quit) 代表退出，**!** 代表强制执行（通常用于覆盖只读警告）。

📊 vi 退出指令对比表

指令 (コマンド)	动作 (日本語)	中文含义	记忆点
:wq 或 ZZ	保存して終了	保存并退出	Write + Quit
:q!	保存せずに終了	强制退出不保存	Quit + ! (不管了)
:w	保存のみ	只保存不退出	Write (写进硬盘)

🧠 小学生级记忆法：“特工脱困记”

想象你是一个被困在文件里的特工：

- 1. **[Esc] (逃生键)**：你在里面写情报（输入模式），突然要撤离。你必须先按 **Esc** (Escape)，意思是“逃跑/撤退”。这样你才能从打字状态变回指挥官状态。
- 2. **[:wq!] (走人命令)**：
 - **:**：拿起对讲机呼叫。

- **w**：先把情报“寄走”(Write)。
 - **q**：然后整个人“撤退”(Quit)。
 - **!**：如果门锁了，就“踹开”它(强制执行)。
3. **[ZZ](睡觉键)**：你也可以直接打两个大写的 **ZZ**。这就好比工作干完了，直接去“睡觉”(ZZZ...)，系统会自动帮你存好并关门。
-

✅ 正确答案

1. **a[Esc]、b[ZZ]**
 2. **a[Esc]、b[:wq!]**
-

📝 考试小秘籍

- **区分大小写**：注意 **ZZ** 必须是大写。如果是小写 **zz**，那是滚动屏幕的命令。
 - **强制符号**：如果题目问“不保存直接退出”，一定要找带 **!** 的选项（如 **:q!**）。
 - **模式切换**：在 vi 的题目中，看到任何动作前没按 **Esc** 或 **:** 的，基本都是陷阱。
-

这一知识点主要考察如何使用 **grep** 命令的 **オプション (选项)** 来进行「**反転検索 (反向搜索)**」以及如何处理「**OR条件 (或条件)**」的匹配。

💡 知识点总结

要提取“不包含”某些字符串的行，核心是使用 **-v** 选项。而要同时匹配多个关键词（如 A 或 B），则需要使用 **拡張正規表現 (扩展正则表达式)** 中的竖线符号 **|**。在 Linux 中，开启扩展正则有两种方式：使用 **grep -E** 或者直接使用 **egrep** 命令。

🔍 选项解析与详细说明

- **✅ `grep -Ev 'PINGT|pingt' test.txt`：正确答案 1。**
 - **理由**：**-E** 开启了扩展正则，使得 **|** 被识别为“或”运算符；**-v** 实现了反转匹配，即排除掉包含这两个词的行。
- **❌ `grep -En 'PINGT|pingt' test.txt`：错误。**
 - **理由**：**-n** 是显示行号。这个命令会搜索并显示包含这两个词的行及其行号，而不是排除它们。
- **✅ `egrep -v 'PINGT|pingt' test.txt`：正确答案 2。**

- 理由: `egrep` 等同于 `grep -E`。配合 `-v` 选项, 它能完美实现排除“PINGT”或“pingt”的要求。
- ✗ `grep -v 'PINGT|pingt' test.txt`: 错误。
 - 理由: 标准的 `grep` 不支持扩展正则。在这种写法下, 它会尝试寻找字面上包含 `|` 这个字符的行, 而不是将其作为“或”逻辑处理。
- ✗ `egrep -n 'PINGT|pingt' test.txt`: 错误。
 - 理由: 同理, 它缺少了关键的 `-v` 选项, 无法实现“不包含”的功能。

`grep` 逻辑组合排查表

需求 (日本語)	必要选项	示例	记忆点
～を含まない	<code>-v</code>	<code>grep -v "A"</code>	Void (排除)
A または B	<code>-E</code> 或 <code>egrep</code>	<code>grep -E "A B"</code>	Extended (扩展)
行番号を表示	<code>-n</code>	<code>grep -n "A"</code>	Number (数字)
ヒット数のみ	<code>-c</code>	<code>grep -c "A"</code>	Count (计数)

小学生级记忆法：“反向过滤网”

想象你有一个装满各种球的箱子, 你想把印有“PINGT”或“pingt”的球全部扔掉:

- `-E` (或者 `egrep`): 这是你的“超级扫描仪”。普通的扫描仪只能看一个词, 超级扫描仪可以同时盯着“左边”和“右边”两个词 (`|`)。
- `-v` (反向开关): 这是你的“排除手套”。平时扫描仪是把抓到的球给你看, 戴上这个手套后, 它会把抓到的球全扔了, 只留下剩下的球。

所以, 想要排除 A 或 B, 你必须同时带上“超级扫描仪”和“排除手套”。

正确答案

- `grep -Ev 'PINGT|pingt' test.txt`
- `egrep -v 'PINGT|pingt' test.txt`

考试小秘籍

- 等价性: 在考试中, 看到 `grep -E` 和 `egrep` 几乎可以划等号。

- **大小写简写**：其实这道题还有一个更聪明的写法是 `grep -vi 'pingt'`。因为 `-i` 可以忽略大小写，这样就不需要写 `|` 符号了。但在多选题中，一定要根据选项给出的逻辑来选。
 - **符号检查**：如果选项里只有 `|` 而没有 `-E` 或 `egrep`，那个选项多半是错的。
-

这一知识点主要考察在 Linux 环境下，如何将「**シェル変数 (本地变量)**」提升为「**環境変数 (环境变量)**」，从而使其能够被当前 Shell 启动的子进程（应用程序或命令）继承并使用。

💡 知识点总结

在 Shell 中，普通定义的变量（如 `VAR=1`）默认只是本地变量，子进程无法访问。要让变量“出圈”变成全局有效的环境变量，需要使用专门的 **定義・変更コマンド (定义/修改命令)**。

🔍 选项解析与详细说明

- ❌ `set`：
 - **理由**：该命令用于列出 **すべての変数 (所有变量)**（包括环境变量和本地变量）以及 Shell 函数。它本身不具备将本地变量转换为环境变量的功能。
 - ❌ `env`：
 - **理由**：该命令主要用于 **表示 (显示)** 环境变量，或者在临时修改的环境中执行命令。它不能永久地将一个已有的本地变量改为环境变量。
 - ❌ `printenv`：
 - **理由**：专门用于 **表示 (显示)** 环境变量。它是一个查看工具，而不是设置工具。
 - ✅ `export`：正确答案 1。
 - **理由**：这是最标准、最常用的命令。它可以直接定义环境变量（`export VAR=val`），也可以将已有的本地变量导出为环境变量（`export VAR`）。
 - ✅ `declare -x`：正确答案 2。
 - **理由**：`declare` 是 Bash 的内置命令，用于声明变量属性。其中 `-x` 选项的作用正是「**後続のコマンドにエクスポートする**」（导出给后续命令），效果等同于 `export`。
-

📊 变量操作命令排查表

コマンド	作用 (日本語)	是否能创建/转换环境变量	核心特点
export	変数をエクスポートする	YES	最常用的标准命令
declare -x	属性を設定してエクスポート	YES	功能丰富，可设置只读等
set	すべての変数を表示	NO	范围最广，包含函数
env	環境変数を表示/一時変更	NO	仅影响紧随其后的命令

🧠 小学生级记忆法：“办理出国护照”

想象变量是住在 “Shell 村” 的村民：

1. **本地变量**：没有护照，只能在村里（当前 Shell）活动。
2. **export (出口/护照办)**：村民去办了一本护照。有了这本护照，他就可以 “**出国**” 到子进程的领地去工作了。
3. **declare -x (带编号的护照)**：这同样是办护照，只是用了更正式的行政手段（**declare** 声明）。

记住：只要看到“让子进程也能用”，就找那个带“出口 (Ex-...)” 含义的词！

✅ 正确答案

1. export
2. declare -x

📝 考试小秘籍

- **-x 的含义**：在 `declare` 命令中，`x` 代表 **eXport**。
- **查看命令区分**：如果题目问“如何**表示**环境变量”，答案是 `env` 或 `printenv`；如果问“如何**作成/变更**”，答案才是 `export`。
- **子进程继承**：环境变量只能 **向下传递**（传给子进程），不能向上产递（子进程修改的变量不会影响父 Shell）。

这一知识点主要考察 Linux 系统引导和运行时自动挂载的核心配置文件 `/etc/fstab` 的结构与逻辑。

💡 知识点总结

`/etc/fstab` 文件是系统挂载的“剧本”。它定义了哪些磁盘分区在开机时自动连接到哪个目录，以及连接时的规则（选项）。理解它的 **項目数 (列数)** 和常用的 **マウントオプション (挂载选项)** 是通过 LPI 考试的关键。

🔍 选项解析与详细说明

- ❌ **項目数は5つである**：错误。
 - 理由：`/etc/fstab` 标准格式包含 **6个项目**（列）。
- ✅ **「mount -a」コマンド実行時にマウントされないマウントオプション指定は「noauto」である**：正确答案 1。
 - 理由：`mount -a` 会挂载配置文件中除了带有 `noauto` 选项以外的所有设备。`noauto` 的字面意思就是“不自动挂载”。
- ❌ **デフォルトのマウントオプションを使用する場合は、マウントオプションの項目は記載してはいけない**：错误。
 - 理由：每一列都必须有内容，不能跳过。如果想使用默认设置，必须在第 4 列明确写上 `defaults`。
- ❌ **デバイス名を指定するのは、UUIDとLABELのみである**：错误。
 - 理由：除了 UUID 和 LABEL，还可以直接使用 **设备文件路径**（如 `/dev/sda1`）。
- ✅ **項目数は6つである**：正确答案 2。
 - 理由：标准的 6 个项目分别是：(1)设备名、(2)挂载点、(3)文件系统类型、(4)选项、(5)dump 标志、(6)fsck检查顺序。
- ❌ **デフォルトのマウントオプションを使用する場合は、リードオンリーでマウントされる**：错误。
 - 理由：`defaults` 选项包含的是 `rw` (read-write, 读写)，而不是只读。

📊 `/etc/fstab` 核心参数排查表

考察点	正确描述	常见陷阱
列数 (項目数)	6列	说是 4 列或 5 列
默认选项	defaults (包含 rw)	说是只读或不用写
自动挂载排除	noauto	说是 none 或 off
设备指定	路径、UUID、LABEL	说只能用其中一种

小学生级记忆法：“六位小管家”

想象 `/etc/fstab` 里住着 6 位小管家，他们排成一排。

1. 管家4 (选项管家)：平时很勤快，你喊一声 `mount -a`，大家就都去干活。但如果他在这本子上写了 `noauto`，他就会在那儿偷懒，不理你的 `mount -a`。
2. 管家数：数一数，正好 6 个人。如果少了一个，这个家（系统）就没法正常开门（挂载）了。

✅ 正确答案

1. 「mount -a」コマンド実行時にマウントされないマウントオプション指定は「noauto」である
2. 項目数は6つである

📝 考试小秘籍

- `mount -a` 的逻辑：它是管理员在修改完 `/etc/fstab` 后最常用的测试命令。如果报错，说明配置文件写错了。
- 默认包含项：记住 `defaults = rw, suid, dev, exec, auto, nouser, async`。其中包含 `auto`，所以它会被 `mount -a` 识别。
- 第 5 和第 6 列：如果记不清是 6 列还是 5 列，想想最后两个数字 `0 0` 或 `1 1`，它们代表了备份和自检，是独立的两个项目。

这一知识点主要考察 **viエディタ (vi 编辑器)** 在未指定文件名启动时的默认行为以及其「**モード (模式)**」的基本逻辑。

💡 知识点总结

在 Linux 命令行中，直接输入 `vi` 而不跟随任何文件名是完全合法的。此时，编辑器会进入一个临时的、未命名的编辑缓冲区。由于没有预设文件名，所有关于“存到哪里”的决定都会推迟到执行保存操作时。

🔍 选项解析与详细说明

- ❌ ファイル名は編集モードに入った際に聞かれる：
 - 理由：进入「入力モード (输入模式)」（如按 `i`）时，vi 不会询问文件名，它只管让你开始打字。
- ✅ 空のファイルで起動する：正确答案 1。
 - 理由：如果不带参数，vi 会打开一个全新的「空のバッファ (空缓冲区)」，屏幕上会显示一列波浪号（`~`），表示这些行目前没有任何内容。
- ❌ 最初にファイル名を指定する画面が出る：
 - 理由：vi 是基于字符终端的工具，没有弹窗或引导界面来让你输入文件名。它直接进入编辑界面。
- ❌ ファイル指定しないと起動できない：
 - 理由：`vi` 命令可以独立运行。这在只是想临时写点东西或者粘贴剪贴板内容时非常方便。
- ✅ ファイル名は保存する際に指定する：正确答案 2。
 - 理由：由于启动时没有指定名字，当你想要退出并保存时，必须使用 `:w <文件名>` 来明确告诉 vi 要把这些内容写到哪个新文件中。
- ❌ 直近の編集ファイルを開いて起動する：
 - 理由：vi 不具备自动记忆并打开上次未关闭文件的功能（除非是由于系统崩溃产生的 `.swp` 恢复文件，但那是另一种机制）。

📊 vi 启动方式对比表

启动命令	状态 (日本語)	文件名指定时机
vi	新規 (空)	保存時 (:w filename)
vi test.txt	既存または新規	起動時 (保存只需 :w)

🧠 小学生级记忆法：“无名笔记本”

想象 vi 是一个路边摊卖的“白纸笔记本”：

1. 直接拿走 (`vi`)：你从摊位上拿了一本没有任何封面的笔记本就开始写。写的时候没人管你叫什么。等你写完了想把它放进书架（硬盘）时，你才发现得在封面上补写一个 **名字**。

2. 买带名字的 (`vi test.txt`): 你买的时候就跟老板说: “我要那本写着 `test.txt` 的本子”。这样你写完直接塞回原位就行了, 不用再写一遍名字。

✅ 正确答案

1. 空のファイルで起動する
2. ファイル名は保存する際に指定する

📝 考试小秘籍

- **保存命令格式:** 记住在没有文件名启动时, 单纯输入 `:wq` 会报错 (E32: No file name), 必须输入 `:w filename`。
- **退出不保存:** 如果你只是练手, 不想取名, 直接用 `:q!` 强制退出即可。
- **界面特征:** 屏幕左侧的一排 `~` (波浪号) 是 vi 空缓冲区的标志性特征。

这一知识点主要考察 Linux 中的「標準出力 (Standard Output)」与「リダイレクト (重定向)」符号的省略规则。

💡 知识点总结

在 Linux 中, 每个打开的文件都有一个 **ファイルデスク립タ (文件描述符)** 数字。

- **1** 代表 **標準出力 (stdout)**, 即命令正常运行的结果。
- **2** 代表 **標準エラー出力 (stderr)**, 即命令报错的信息。
- 当我们在进行重定向 (`>` 或 `>>`) 时, 如果省略前面的数字, 系统默认会认为你在处理 **标准输出 (1)**。

🔍 选项解析与详细说明

我们先来拆解题目中的五个选项:

- A. `コマンド > ファイル`: 将标准输出 **上書き (覆盖)** 到文件。
- B. `コマンド >> ファイル`: 将标准输出 **追記 (追加)** 到文件末尾。
- C. `コマンド 1>> ファイル`: 显式地将标准输出 (1) **追記** 到文件。
- D. `コマンド 1> ファイル`: 显式地将标准输出 (1) **覆盖** 到文件。
- E. `コマンド 2> ファイル`: 将标准 **错误** 输出 (2) 覆盖到文件。

组合匹配:

- 1. **A 和 D**：两者都是“覆盖”模式。由于 `>` 默认就是 `1>`，所以它们完全等价。
- 2. **B 和 C**：两者都是“追加”模式。由于 `>>` 默认就是 `1>>`，所以它们完全等价。

 重定向符号简写对照表

完整写法	省略写法	动作 (日本語)	效果
<code>1> file</code>	<code>> file</code>	上書き (Overwrite)	清空原文件，写入新内容
<code>1>> file</code>	<code>>> file</code>	追記 (Append)	保留原内容，在新行追加
<code>2> file</code>	(不可省略)	エラー出力	仅保存报错信息

 小学生级记忆法：“隐身的 1 号”

想象每个命令都有两个口袋：

- **1 号口袋**：装的是做好的手办（正常结果）。
- **2 号口袋**：装的是碎掉的零件（报错信息）。

当你下令“把口袋里的东西倒进盒子里”时：

- 1. 如果你不说是哪个口袋（直接用 `>` 或 `>>`），大家默认都会去倒 **1 号口袋**。
- 2. 只有你大喊“**2 号！**”时，大家才会去处理那个装碎零件的口袋。

所以，带 `1` 的和不带数字的，其实干的是同一件事。

 正确答案

- 1. **A と D**
- 2. **B と C**

 考试小秘籍

- **数字 1 的默认性**：在 LPI 考试中，只要看到 `>` 前面没有数字，脑子里立刻补上一个隐形的 `1`。
- **覆盖 vs 追加**：一个箭头 `>` 是覆盖（把旧的踢走），两个箭头 `>>` 是追加（乖乖排队）。
- **区分 2**：记住 `2>` 永远不能省略数字，否则就变成处理标准输出了。

这一知识点主要考察 **パイプ (管道 |)** 与 **tee コマンド** 的配合使用，以及数据在命令链条中的流向。

💡 知识点总结

- **パイプ (|)**: 将前一个命令的 **標準出力** 传递给后一个命令的 **標準入力**。
- **tee コマンド**: 像三通水管一样，将收到的数据同时发往两个地方：一个是 **ファイル (文件)**，另一个是 **標準出力**。
- 如果 **tee** 后面没有跟文件名，它依然会将收到的数据通过标准输出传给下一个管道。

🔍 选项解析与详细说明

- ☒ 「ping-t」が表示される：正确答案。
 - **理由**:
 - i. `echo ping-t` 输出字符串 `ping-t`。
 - ii. 这个字符串通过管道传给 `tee`。由于 `tee` 没指定文件，它只负责把收到的 `ping-t` 原样传给下一个管道。
 - iii. 最后 `cat` 接收到 `ping-t` 并将其打印在屏幕上。
- ☒ 何も表示されない：错误。
 - **理由**: 数据流并没有被截断，`tee` 和 `cat` 都会正常处理并输出内容。
- ☒ 「tee」が表示される：错误。
 - **理由**: `tee` 在这里是命令名，不是要输出的文本内容。
- ☒ catコマンドの入力待ち状態になる：错误。
 - **理由**: `cat` 已经通过管道接收到了来自 `tee` 的数据，所以它会立即处理并结束，不会进入等待手动输入的模式。
- ☒ 「echo ping-t | tee」が表示される：错误。
 - **理由**: Shell 会解释执行这些符号，而不是把它们当成普通字符串打印出来。

tee 命令的数据流向图

步骤	命令	输入数据	输出数据 (给下一级)
1	echo ping-t	(无)	ping-t
2	`	` (管道)	ping-t
3	tee	ping-t	ping-t
4	cat	ping-t	ping-t (屏幕显示)

小学生级记忆法：“三通水管”

想象数据是水管里的「水」：

- echo**：是水龙头，放出了名为 `ping-t` 的水。
- tee**：是一个「T型三通接头」。正常情况下，它一边把水接到桶里（文件），一边让水继续往前流。
- 本题情况**：这次 `tee` 没接桶，所以水直接顺着管道流到了最后的 **cat**（排水口）里喷了出来。最后你在排水口看到的当然还是最初放出来的那些水：`ping-t`。

✓ 正确答案

「ping-t」が表示される

✎ 考试小秘籍

- tee 的真本事**：考试中常考 `echo "hello" | tee file.txt`。这时候屏幕会显示 `hello`，同时 `file.txt` 里也会写入 `hello`。
- 管道连续性**：只要中间的命令（如 `tee` 或 `grep`）能产生标准输出，管道就能一直连下去。
- 参数缺失**：看到 `tee` 后面没带参数不要慌，它只是变成了一个“透明的转发站”。

这一知识点主要考察 Linux Shell 中「エスケープ (转义)」符号的使用，以及它与「チルダ (~)」展开之间的优先级关系。

💡 知识点总结

在 Linux 中，反斜杠 `\` 是一个特殊字符，用于取消紧随其后的字符的特殊含义。

- 目标**：创建一个名为 `\work` 的目录。

- **关键点：**由于 `\` 本身有特殊含义，如果你想让目录名真的包含一个反斜杠，你必须输入两个反斜杠 `\\`，或者用引号保护它。
 - **チルダ (~)：**代表当前用户的ホームディレクトリ。**注意：**如果把 `~` 放在引号里，它就会失去“自动展开”的功能，变成一个普通的字符。
-

选项解析与详细说明

- **✗ `mkdir ~\\work`：**
 - **理由：**Shell 会将 `\\` 解释为一个普通的 `\`。但由于没有 `/` 分隔符，系统会尝试寻找名为 `~` 的用户并访问其 `\work` 目录，这通常会导致“用户不存在”的错误。
 - **✗ `mkdir "~/\work"`：**
 - **理由：**双引号会阻止 `~` 的展开。系统会尝试在当前目录下创建一个字面上叫 `~/\work` 的目录，而不是在主目录下创建。
 - **✓ `mkdir ~/\work`：正确答案。**
 - **理由：**
 - `~` 不在引号内，正确展开为 `/home/user`。
 - `/` 作为路径分隔符。
 - `\\` 经过 Shell 转义后，变成一个普通的字符 `\`。
 - 最终效果是在主目录下创建 `\work`。
 - **✗ `mkdir '~/\work'`：**
 - **理由：**单引号是最严格的保护，它会让里面所有的字符（包括 `~` 和 `\\`）都变成普通字面量。结果会创建一个名字极其奇怪的目录。
 - **✗ `mkdir ~/\work`：**
 - **理由：**单个 `\` 会转义后面的 `w`。因为 `w` 本身没特殊含义，转义后还是 `w`。结果只是创建了一个普通的 `~/work` 目录，没有反斜杠。
-

Shell 特殊字符转义对照表

写法	含义 (日本語)	最终文件名结果
\\	バックスラッシュ自体	\
\n	改行	(换行符)
"~"	文字としてのチルダ	~ (不展开)
~	ホームディレクトリ	/home/ username

小学生级记忆法：“反斜杠的变身术”

想象反斜杠 `\` 是一个「隐身斗篷」：

- 如果你只穿一件 (`\w`)：斗篷盖住了 `w`。但 `w` 本来就是普通人，穿不穿都一样，最后还是 `w`。
- 如果你穿两件 (`\\`)：第一件斗篷盖住了第二件。结果第二件斗篷被迫“显形”了！你终于在文件名里看到了那个 `\`。
- 注意 `~` (传送门)：`~` 是一个“传送门”，能让你带回家。但如果你把它关在“引号笼子”里，传送门就失效了，它只是一把普通的梯子。

所以，想要“回家”并在那里建个带“梯子”的房子，必须是：**传送门 + / + 双层斗篷**。

正确答案

`mkdir ~/\work`

考试小秘籍

- 引号的陷阱：看到 `~` 出现在引号里，直接判定它不会变成 `/home/xxx`。
- 转义规则：想要在文件名里看到 `\`、`*`、`?` 等特殊符号，最稳妥的方法就是前面加个 `\` 或者用单引号。
- 创建多级目录：如果题目问如何一次性创建 `a/b/c`，记得找带 `-p` (parents) 选项的答案。

这一知识点主要考察 Linux 与 Windows 之间「改行コード (换行符)」的差异。这是在跨平台开发或系统迁移时最常遇到的“乱码/解析错误”问题。

知识点总结

不同操作系统的文本文件对“换行”的定义不同：

- **Windows:** 使用 CRLF (`\r\n`), 即 “回车 + 换行”。
- **Linux/Unix:** 使用 LF (`\n`), 即只有 “换行”。
- 如果在 Linux 上直接运行或处理 Windows 创建的文本（尤其是脚本文件），多出来的 CR (`\r`) 字符常会被识别为非法字符（显示为 `^M` ），导致程序报错。因此，必须将 `\r` 删掉。

 选项解析与详细说明

- **✗ テキストファイルから改行コードLF (`\n`) を取り除く:**
 - **理由:** LF 是 Linux 唯一识别的换行标志，删了它文件就变成一行长长的字符串了。
- **✗ テキストファイルの改行コードをLFCR (`\n\r`) にする:**
 - **理由:** 不存在这种标准顺序的换行符。
- **✗ テキストファイルの改行コードをCRLF (`\r\n`) にする:**
 - **理由:** 这是 Windows 的标准。题目要求是在 Linux 下正しく扱える（正确处理），所以目标应该是转换成 LF。
- **✓ テキストファイルから改行コードCR (`\r`) を取り除く: 正确答案。**
 - **理由:** 将 Windows 的 `\r\n` 去掉 `\r` 后，就变成了 Linux 完美的 `\n`。
- **✗ そのままで問題なく扱える:**
 - **理由:** 虽然普通的 `cat` 查看可能没问题，但如果是 Shell 脚本或配置文件，多出来的 `\r` 会导致执行失败。

 改行コード（换行符）对照表

操作系统	缩写	转义字符	十六进制 (Hex)
Linux / Unix	LF	<code>\n</code>	0A
Windows	CRLF	<code>\r\n</code>	0D 0A
Legacy Mac	CR	<code>\r</code>	0D

 小学生级记忆法：“打字机的动作”

想象老式打字机换行的动作：

1. **CR (Carriage Return):** 把打印头推回最左边（回车）。
2. **LF (Line Feed):** 把纸向上卷一行（换行）。

- **Windows 很守旧**：它坚持要先回车、再换行（两个动作都要）。
- **Linux 很激进**：它觉得纸往上卷了，打印头自然就该在下一行开始。所以它只需要换行 (LF)，把多余的“回车 (CR)”视为累赘。

所以，进 Linux 的家门，得把 Windows 带过来的那个“回车 (CR)”脱掉丢进垃圾桶。

✅ 正确答案

テキストファイルから改行コードCR (\n) を取り除く

✎ 考试小秘籍

- **常用转换工具**：考试可能会考到相关的转换命令，记住 `dos2unix`（Windows转Linux）和 `unix2dos`（Linux转Windows）。
- **tr 命令妙用**：也可以用 `tr -d '\r' < windows.txt > linux.txt` 来手动删除。
- **vi 中的表现**：如果在 vi 里看到每行末尾都有个奇怪的 `^M`，那就是多余的 CR 字符在作怪。

オプション	説明
-d	文字列1で指定した文字を削除
-s	文字列1で指定した文字が連続した場合、1文字に置き換える

这一知识点主要考察如何使用 `tr` (Translate) 命令来处理「改行コード (换行符)」的差异。在 Linux 环境下，处理来自 Windows 的 **CRLF** 文件，核心目标是删除多余的 **CR (\r)**。

💡 知识点总结

`tr` 命令用于 **文字の変換 (转换)** 或 **削除 (删除)**。

- **-d 选项**：代表 **Delete**，用于删除指定的字符。
- **目标**：Windows 的换行符是 `\r\n` (CRLF)，Linux 只需要 `\n` (LF)。因此，必须 **删除 \r**。
- **数据流**：`tr` 不直接接受文件名作为参数，必须通过 **リダイレクト (<)** 或 **パイプ (|)** 将内容传给它。

🔍 选项解析与详细说明

- **✅ cat crlf.txt | tr -d '\r' > linux.txt**：正确答案 1

- **理由：**使用 `cat` 将文件内容喷出，通过管道 `|` 传给 `tr`，`tr -d '\r'` 负责把所有的回车符 (`\r`) 删掉，最后重定向到新文件。虽然多写了一个 `cat`，但在 Linux 中是完全正确且常用的写法。
-  `tr -d '\r' < crlf.txt > linux.txt`：正确答案 2
 - **理由：**这是最标准、最高效的写法。直接使用输入重定向 `<` 将文件内容喂给 `tr`，并删除 `\r`。
-  `tr '\r' '' < crlf.txt > linux.txt`
 - **理由：**`tr` 的转换模式要求两个参数长度对等。如果不加 `-d`，它会尝试把 `\r` 替换成空，但这在 `tr` 的基本语法中是不成立的（会报错或提示参数不足）。
-  `cat crlf.txt | tr -d '\n' > linux.txt`
 - **理由：**删错了！`\n` 是换行符。如果把它删了，整个文件所有的行都会挤在一起，变成一个巨大的单行字符串。
-  `tr -d '\n' < crlf.txt > linux.txt`
 - **理由：**同上，这会破坏文件的行结构。

`tr` 命令常用操作排查表

操作需求	命令格式	作用 (日本語)
删除 CR (<code>\r</code>)	<code>tr -d '\r'</code>	Windows 格式 转 Linux (删除回车)
删除 LF (<code>\n</code>)	<code>tr -d '\n'</code>	行を連結 (把所有行连成一行)
大小写转换	<code>tr 'a-z' 'A-Z'</code>	小文字を大文字 に変換
重复压缩	<code>tr -s ''</code>	連続するスペースを1つに (压缩空格)

小学生级记忆法：“橡皮擦与垃圾桶”

想象 `tr -d` 是一个专门负责「擦除」的橡皮擦：

1. 我们要擦什么？Windows 换行符里那个多出来的 `\r` 就像是作业本上的污点。
2. 怎么擦？
 - **方法一 (水管流)：**用 `cat` 把水放出来，经过带有 `tr -d '\r'` 滤网的管道。

- 方法二 (漏斗流): 用 `<` 漏斗把文件倒进 `tr -d '\r'` 这个清洁机器里。

3. 千万别擦 `\n`: `\n` 是书页的折痕。要是把折痕 (`\n`) 擦了, 整本书就卷成一个纸团了!

✅ 正确答案

1. `cat crlf.txt | tr -d '\r' > linux.txt`
2. `tr -d '\r' < crlf.txt > linux.txt`

📝 考试小秘籍

- 参数位置: 记住 `tr` 后面 **不能** 直接跟文件名 (如 `tr -d '\r' file.txt` 是错误的)。
- 单引号: 在指定 `\r` 或 `\n` 时, 建议加上单引号, 防止 Shell 对反斜杠产生误解。
- 配套工具: 除了 `tr`, 考试中如果出现 `sed -i 's/\r//' filename`, 它也能实现相同的功能, 且可以直接修改原文件。

ユーザーIDを表示させるシェルスクリプトを作成し実行したところ、以下のエラーが表示された。

```
$ sh userid.sh
userid.sh: 行 2: $'¥r': コマンドが見つかりません
userid.sh: 行 8: 構文エラー: 予期しないファイル終了 (EOF) です
```

考えられる原因はなにか

- ☐ シェルスクリプトに実行権限がついていない
- ☒ 改行コードの異なるシステムで作成されたファイルを実行した
- ☐ `\r` コマンドをインストールしていない
- ☐ 一般ユーザーで実行している

这一错误现象是 Linux 初学者在处理跨平台文件时最常遇到的典型问题。

💡 知识点总结

报错信息中的 `$'\r': コマンドが見つかりません` 是核心线索。在 Linux 系统中, 脚本执行器会将 Windows 风格的换行符 **CRLF** (`\r\n`) 中的 **CR** (`\r`) 误认为是一个普通的字符或命令。由于系统中不存在名为 `\r` 的命令, 因此会触发“未找到命令”的报错。

🔍 选项解析与详细说明

- ❌ **シェルスクリプトに実行権限がついていない:**
 - 理由: 如果缺少执行权限, 报错通常是 `Permission denied` (许可被拒绝)。而且题目中是使用 `sh userid.sh` 运行的, 这种方式不需要脚本具备执行权限。
- ✅ **改行コードの異なるシステムで作成されたファイルを実行した: 正确答案。**
 - 理由: 图片中的报错显示了 `$'\r'`。这明确表示该脚本是在 **Windows** (使用 CRLF) 上编写后直接拷贝到 **Linux** (使用 LF) 运行的。Linux 无法识别行末多出来的 `\r` (回车符), 导致脚本逻辑断裂并报错。
- ❌ **\rコマンドをインストールしていない:**
 - 理由: `\r` 不是一个真实存在的 Linux 命令, 它是换行符的一部分。不存在安装这个命令的说法。
- ❌ **一般ユーザーで実行している:**
 - 理由: 权限不足的报错通常与 `sudo` 或文件所属有关。这种“找不到命令”且包含特殊转义符的报错与用户身份无关。

 处理“设备忙”或“执行错误”的排查表

报错信息关键词	可能原因	解决方法
<code>\$'\r'</code> 或 <code>^M</code>	Windows 换行符 (CRLF)	使用 <code>dos2unix</code> 或 <code>tr -d '\r'</code> 转换
<code>Permission denied</code>	权限不足 / 没加 <code>+x</code>	<code>chmod +x</code> 或使用 <code>sh</code> 执行
<code>Syntax error: EOF</code>	括号未闭合 / 换行符干扰	检查脚本逻辑并转换换行符

 小学生级记忆法：“隐形的骨头”

想象 Linux 是一只只吃「肉 (LF)」的小狗。

- Windows 的肉:** 每一块肉后面都连着一根「骨头 (`\r`)」。
- 报错原因:** Linux 小狗咬到肉的时候, 突然被那根看不见的骨头卡住了嗓子。它大喊: “我不认识这根骨头 (`\r`), 这不是我要的命令!”
- 解决方法:** 把骨头剔掉 (转换成 Linux 的 LF 格式), 小狗就能开心地吃肉了。

✅ 正确答案

改行コードの異なるシステムで作成されたファイルを実行した

✎ 考试小秘籍

- **识别标识符**：看到 `\r`、`^M` 或者 `0d` (Hex)，脑子里要立刻反映出“Windows 换行符”。
- **修正命令**：记住修正这个问题的两个常用工具：`tr -d '\r'` 和 `dos2unix`。
- **EOF 错误**：有时由于 `\r` 的存在，Shell 无法正确找到脚本的结尾，也会报图片中显示的 `予期しないファイル終了 (EOF)` 错误。

コマンド履歴保存ファイルに記録が残らないように以下のとおり作業を行った。

\$

空欄に当てはまるものはどれか

- ☐ COMMAND
- ☐ HISTSAVE
- ☐ HISTORY
- ☒ HISTFILESIZE
- ☐ HISTSIZE

这道题目考察的是 Linux Bash 中控制「**コマンド履歴 (命令历史)**」的环境变量设置，以及跨平台脚本运行时的常见错误。

💡 知识点总结

- **命令历史控制**：Bash 使用变量来限制保存的历史记录条数。若要彻底不留痕迹，需要同时控制 **内存中** 的记录和 **硬盘文件** 中的记录。
- **换行符差异**：Windows 使用 **CRLF** (`\r\n`)，Linux 使用 **LF** (`\n`)。在脚本中多出的 `\r` 会被 Linux 误认为是异常命令。

🔍 选项解析与详细说明

针对填空题 (`$ ____ = 0`)：

- **✗ COMMAND / HISTSAVE / HISTORY**：
 - **理由**：这些都不是 Bash 预定义的用于控制历史记录数量的环境变量。
- **✓ HISTFILESIZE：正确答案。**
 - **理由**：控制 **历史文件** (`.bash_history`) 中能够保存的最大行数。设置为 0 后，退出 Shell 时将不会有任何记录被写入硬盘。
- **✓ HISTSIZE：正确答案。**

- 理由：控制 当前会话内存 中保存的命令条数。设置为 0 后，当前输入的命令将不会被记录在缓存中。

处理“执行错误”与“历史记录”的排查表

场景	关键现象/变量	解决方法
脚本执行报错	<code>\$'\r': コマンドが見つかりません</code>	将文件换行符从 CRLF 转为 LF
内存不留历史	<code>HISTSIZE=0</code>	禁止当前会话记录命令
硬盘文件不留记录	<code>HISTFILESIZE=0</code>	禁止将记录写入 <code>.bash_history</code>

小学生级记忆法：“隐形特工的笔记”

想象你在执行秘密任务：

- `HISTSIZE` (大脑记忆)：设为 0 意味着你“瞬间失忆”，刚下达的指令脑子里立刻就不记得了。
- `HISTFILESIZE` (秘密档案袋)：设为 0 意味着“档案袋漏了”，就算你脑子里记得（内存有数据），下班回家往袋子里装的时候也会全部掉出来，文件里永远是空的。
- `$'\r'` 报错 (卡住的骨头)：你从 Windows 带过来的指令后面都带着一根“隐形的骨头 (`\r`)”。Linux 嗓子细，咽不下这根骨头，所以会大喊“找不到这个命令”！

正确答案

- `HISTFILESIZE`
- `HISTSIZE`

考试小秘籍

- 报错识别：看到报错信息里有 `\r` 或 `^M`，100% 是换行符格式问题。
- 变量区分：`FILESIZE` 影响的是硬盘上的 文件；没有 `FILE` 的那个只影响 内存。
- 组合拳：在实际工作中，通常需要同时设置这两个变量为 0 才能确保隐私。

看来这道题的考官非常严谨，他精准地抓住了题目中的**“保存文件 (保存ファイル)”**这四个字。在 LPI 考试中，虽然 `HISTSIZE` 和 `HISTFILESIZE` 经常成对出现，但它们分工非常明确：

知识点总结

- **HISTSIZE**：控制 **内存 (Memory)** 中的缓冲区大小。它决定了你输入 `history` 命令时能看到多少条记录。
- **HISTFILESIZE**：控制 **磁盘文件 (Disk File)** 的大小。它专门负责 `.bash_history` 这个文件的上限。

由于题目明确提到了“**コマンド履歴保存ファイル**”（命令历史保存文件），所以最核心、最直接的答案确实是针对文件的那个变量。

选项解析与详细说明

针对填空题（\$ ____=0）：

-  **HISTFILESIZE：唯一最准确答案。**
 - **理由：** 题目设定的目标是“文件里不留记录”。即使 `HISTSIZE` 很大（内存记住了），只要 `HISTFILESIZE=0`，在用户登出（Logout）时，Bash 准备往文件里写数据时就会发现“容量限制为 0”，从而导致文件内容被清空或无法写入。
-  **HISTSIZE**：虽然它也能达到间接效果（内存都不记了，文件自然也没东西写），但它不属于直接控制“保存文件”容量的开关。

历史记录变量精准排查表

变量名	目标对象 (日本語)	关键职能	考试辨析点
HISTFILESIZE	保存ファイル	限制 .bash_history 的行数	看到 “ファイル” 字样必选
HISTSIZE	メモリ (バッファ)	限制当前窗口 history 的显示数	看到 “表示件数” 或 “メモリ” 时选

小学生级记忆法：“漏斗与储物箱”

1. **HISTSIZE** 是“漏斗”：控制你能抓多少数据。
2. **HISTFILESIZE** 是“储物箱”：控制最后能塞进仓库（文件）多少东西。

题目说的是“**仓库（文件）里不要留东西**”，所以我们直接把“**储物箱 (FILESIZE)**”的容量封死为 0。哪怕漏斗抓了再多数据，最后仓库里也一两都没有！

正确答案

HISTFILESIZE

📝 考试小秘籍

- **抓关键字：**LPI 考试非常喜欢玩这种微小的文字游戏。看到 “**ファイル (File)**” 就要条件反射找带 “**FILE**” 的变量。
- **实战验证：**如果你在 Linux 输入 `HISTFILESIZE=0` 然后退出再登录，你会发现原本的 `.bash_history` 里的内容全被擦除了。

这张图展示了 `HISTFILESIZE=0` 这一设置如何影响 Linux 命令历史记录的 **持久化保存**。

🔍 选项解析与详细说明

这张长图可以分为 **三个阶段** 来理解：

第一阶段：正常状态（未设置变量前）

- 用户输入了 `history`、`ls` 等命令，并查看了 `$HISTFILE`。
- 执行 `exit` 退出并重新登录后：再次输入 `history`，可以看到之前的记录依然存在。
- **结论：**这是默认行为，系统会自动把内存里的记录同步到硬盘。

第二阶段：设置变量 `HISTFILESIZE=0`

- 图中红色部分显示执行了 `HISTFILESIZE=0`。
- 注意此时的 `history` 结果：第 1 到第 9 条记录依然清晰可见。
- **关键理解：**设置 `HISTFILESIZE=0` **不会立即抹除**你当前内存里的记忆，它只是把“档案袋”封死了。

第三阶段：退出并验证结果

- 用户执行 `exit` 登出，然后 **再次登录**。
- 重新登录后的 `history` 结果：只剩下刚刚输入的第 1 条 `history`。之前的 `ls`、`cat`、甚至 `HISTFILESIZE=0` 这一条本身都消失了！

```
$ history
1 history
2 ls -l $HISTFILE
3 history
$ cat $HISTFILE
$ exit
(ログアウト後、再ログイン)
$ history
1 history
2 ls -l $HISTFILE
3 history
4 cat $HISTFILE
5 exit
6 history
$ cat $HISTFILE
history
ls -l $HISTFILE
history
cat $HISTFILE
exit
$ HISTFILESIZE=0
$ history
1 history
2 ls -l $HISTFILE
3 history
4 cat $HISTFILE
5 exit
6 history
7 cat $HISTFILE
8 HISTFILESIZE=0
9 history
$ exit
(ログアウト後、再ログイン)
$ history
1 history
```

前回ログイン時点の履歴が残っている

ログアウト前の履歴が残っている

履歴が残っていない

- **结论：**因为设置了 `HISTFILESIZE=0`，在用户退出的一瞬间，Bash 发现“硬盘档案袋”的容量是 0，于是拒绝写入任何数据。下次登录时，系统从空的档案袋里读不到任何旧记录。

💡 知识点总结

Bash 命令历史记录的保存分为两个阶段：

1. **内存阶段：**当前正在输入的命令暂存在内存缓冲区（由 `HISTSIZE` 控制）。
2. **硬盘阶段：**当你退出登录（`exit`）时，Bash 会将内存中的记录写入硬盘上的文件（由 `HISTFILESIZE` 控制，默认文件是 `~/.bash_history`）。

📊 处理“设备忙”问题的排查表

（在这里我们借用排查表形式对比两个核心变量的“工作现场”）

变量名	影响时间点	影响位置	现象
HISTSIZE	立即生效	内存	设置为 0 后，再输入 history 结果立刻变空。
HISTFILESIZE	退出登录后	硬盘文件	当前能看到记录，但重启/重连后记录全丢。

🧠 小学生级记忆法：“放学后的清空令”

想象你在学校上一天的课：

- `HISTSIZE` 是你的“脑容量”。
- `HISTFILESIZE` 是你家里的“作业柜”。

图中的操作就像是：你白天在学校记了一脑子的笔记（内存有数据）。放学前你下令：“家里的作业柜容量设为 0 (`HISTFILESIZE=0`)”。

虽然你回家路上还记得学校的事，但当你回到家（`exit` 退出），想把脑子里的东西写进柜子时，发现柜子锁死根本放不进去。第二天早上你再回学校（重新登录），脑子清空了，柜子里也空的，你就彻底“断片”了。

✅ 正确答案

这张图演示的是：通过设置 `HISTFILESIZE=0`，使命令历史在退出登录后无法保存到文件，从而达到“再次登录时历史记录为空”的效果。

 考试小秘籍

- **观察图中的 `exit`**：这是关键！如果题目图中没有出现 `exit` 而历史记录消失了，那是 `HISTSIZE` 的功劳；如果出现了 `exit` 重启后才消失，那就是 `HISTFILESIZE` 的作用。
- **黄色注释识别**：图中黄色字写着“前回ログイン時点の履歴が残っている”（保留了上次登录的记录）和“履歴が残っていない”（记录未保留），这正是对比实验的核心。

環境変数	説明
HISTFILE	コマンド履歴保存ファイルのパス
HISTSIZE	現在のシェルでのコマンド履歴の保存数
HISTFILESIZE	コマンド履歴保存ファイルへの履歴保存数
HOSTNAME	ホスト名
HOME	ログインしているユーザのホームディレクトリ
LANG	ロケール(言語設定)
PATH	コマンドやプログラムを検索するディレクトリのリスト
PWD	カレントディレクトリのパス
USER	ログインしているユーザ

太棒了，结合你提供的所有图片，我为你整理了一份专门针对 **LPI 考试** 的“通关背诵表”。这份表格不仅涵盖了环境变数，还把图片中出现的那个报错（`\r`）也一并收录了。

 知识点总结

这份资料的核心在于区分“**内存 vs 硬盘**”以及“**Windows vs Linux 规范**”。理解了这两对矛盾，你就掌握了考试的命门。

 环境变量与核心报错背诵表

分类	关键术语 (日本語)	说明与考点 (中文)	记忆钩子
历史记录	HISTSIZE	メモリ上の保存数。影响当前窗口 history 的显示。	Size = Screen (屏幕能看到的)
历史记录	HISTFILESIZE	保存ファイルへの保存数。影响写入磁盘的数量。	File = Forever (永久保存的)
历史记录	HISTFILE	履歴保存ファイルのパス (默认为 ~/.bash_history)。	文件名本身
用户信息	HOME / USER	ユーザのホームディレクトリ / ログインユーザ名。	基础常识
系统环境	HOSTNAME	ホスト名。当前机器的名字。	机器名
系统环境	PATH	コマンドを検索するディレクトリのリスト。	寻找命令的路线图
位置状态	PWD	カレントディレクトリのパス。	Present (当前所在)
语言区域	LANG	ロケール (言語設定)。控制显示语言。	Language (语言)
典型报错	<code>\$'\r' / ^M</code>	改行コード (CRLF) 导致的错误。多出现在 Windows 编写的脚本中。	嗓子里的“骨头”

实验逻辑排查表

根据你提供的长图，我们可以总结出这个实验的三个关键状态：

动作	内存状态 (history 命令)	文件状态 (cat \$HISTFILE)	下次登录后
默认状态	有记录	有记录	记录依然存在
设 HISTFILESIZE=0	仍有记录 (内存没清)	文件被清空/无法写入	记录全丢 (只剩 1 条)
设 HISTSIZE=0	立即变空	文件取决于退出前的状态	记录全丢

小学生级记忆法

- HISTFILESIZE=0：就像是“档案袋底部漏了”。你在教室里（内存）记了一整天笔记，但放学往袋子里装的时候全掉光了，第二天回学校一看，包里是空的。
- HISTSIZE=0：就像是“瞬间失忆”。你刚听完一句话立刻就忘了，根本等不到放学。
- `\r` 报错：它是 Windows 偷偷塞给你的“隐形累赘”。Linux 这个“洁癖小哥”不认识它，所以会气得罢工报错。

 考试小秘籍

- 1. 看到“ファイル”选 HISTFILESIZE：考试题干中如果出现“ファイル (File)”字样，它是答案的概率是 99%。
- 2. 看到“\r”报错找 CRLF：这是固定搭配。解决方法一定是 `tr -d '\r'` 或 `dos2unix`。
- 3. 变量全大写：记住所有的环境变量在 Linux 里面默认都是全大写的，小写在考试中是干扰项。

这一知识点主要考察 Linux 在线手册 **manページ (man手册)** 的 **セクション番号 (章节编号)** 分类逻辑。

 知识点总结

Linux 的 `man` 手册将不同类型的命令、文件和库函数分门别类地存放在不同的 **セクション (章节)** 中。当我们需要查看特定类型的帮助信息（例如区分 `passwd` 命令和 `/etc/passwd` 配置文件）时，指定章节编号至关重要。

 选项解析与详细说明

根据题目，`shutdown` 是一个用于关闭或重启系统的命令，它涉及到系统的底层运行状态，通常需要 **root 权限** 才能执行。

- ✗ 1 (ユーザーコマンド): 存放普通用户可以执行的常用命令（如 `ls` , `cat` ）。
- ✗ 2 (システムコール): 存放内核提供的函数。
- ✗ 3 (ライブラリ呼び出し): 存放程序库中的函数。
- ✗ 4 (特殊ファイル): 存放 `/dev` 目录下的设备文件信息。
- ✗ 5 (ファイルの書式と慣習): 存放配置文件的格式（如 `/etc/passwd` , `/etc/fstab` ）。
- ✗ 6 (ゲーム): 存放游戏相关信息。
- ✗ 7 (その他いろいろなもの): 存放杂项（如宏包或惯例）。
- ✓ 8 (システム管理コマンド): 正确答案。

セクション番号	内容
1	ユーザーコマンド
2	システムコール (カーネルが提供する関数)
3	ライブラリ呼び出し (プログラムライブラリに含まれる関数)
4	特殊ファイル (通常 /dev 配下に存在するファイル)
5	ファイルの書式と慣習 (例: /etc/passwd)
6	ゲーム
7	その他いろいろなもの (マクロパッケージや慣習などを含む) 例えば man(7) や groff(7)
8	システム管理コマンド (通常は root 用)
9	カーネルルーチン [非標準]

- 理由：该章节专门存放 **系统管理命令**，这些命令通常供 **root 用户** 使用。由于 `shutdown` 是典型的系统管理工具，它被归类在第 8 部分。
- ✗ 9 (カーネルルーチン)**：存放非标准的内核例程。

 man 页面核心章节排查表

章节编号	内容 (日本語)	常见考点示例	区分技巧
1	ユーザーコマンド	ls, cp, passwd(命令)	普通人用的工具
5	ファイルの書式	/etc/passwd, fstab	只是配置文本，不能执行
8	システム管理	shutdown, ifconfig, reboot	root 大神用的工具

 小学生级记忆法：“管理者的 8 号勋章”

想象 `man` 手册是一座 8 层的图书馆：

- 1 楼 (普通区)**：谁都能进，大家都在这里拿 `ls` 扫帚打扫卫生。
- 5 楼 (图纸区)**：放的是各种房子的图纸（配置文件格式），只能看不能动。
- 8 楼 (超级管理员办公室)**：只有拿着大钥匙（root 权限）的管理员能进。这里放着能够关掉整个大楼电源的 `shutdown` 按钮。

记住：超级管理 (Admin) 的动作都在最高的 8 楼！

 正确答案

8

 考试小秘籍

- 同名干扰项**：考试最常考 `passwd`。`man 1 passwd` 看的是改密码的命令；`man 5 passwd` 看的是用户账户文件的格式。

- **快速搜索**：如果忘了命令在哪个章节，可以用 `man -k` (等同于 `apropos`) 搜索关键词，它会列出所有相关的章节。
- **精确匹配**：使用 `man -f` (等同于 `whatis`) 可以显示命令的简短描述及其所属章节。

オプション	説明
-k	キーワードに一部一致するコマンドやファイルのmanページを一覧表示 (aproposコマンドと同じ)
-f	キーワードに完全一致するコマンドやファイルのmanページを一覧表示 (whatisコマンドと同じ)

执行不带任何参数或选项的 `mount` 命令是 Linux 系统管理中的基础操作。

💡 知识点总结

当 `mount` 命令单独执行（不指定设备或挂载点）时，它的作用是读取内核中的挂载信息，并将当前系统已成功挂载的所有文件系统状态呈现给用户。

🔍 选项解析与详细说明

- ❌ 「`/etc/fstab`」ファイルの内容を反映する：
 - 理由： `mount` 命令本身不具备自动“同步”或“反映”配置文件的功能。
- ❌ 不正な使い方であり、エラーになる：
 - 理由：这是完全合法的用法。它被设计为一种查询当前挂载状态的快捷方式。
- ❌ 「`/etc/fstab`」ファイルに書かれているファイルシステムをすべてマウント：
 - 理由：如果要根据 `/etc/fstab` 自动挂载所有内容，必须使用 `mount -a` (all) 命令。
- ✅ 現在マウントされているファイルシステムの一覧を表示：正确答案。
 - 理由：不带参数运行 `mount` 会列出当前所有已挂载的分区、设备路径、挂载点以及使用的挂载选项（如 `rw`，`relatime` 等）。这等同于查看 `/proc/mounts` 文件的内容。
- ❌ 「`/etc/fstab`」ファイルの内容を表示：
 - 理由：要查看该文件的内容，应使用 `cat /etc/fstab`。虽然 `mount` 的结果与此文件有关联，但两者显示的内容维度不同（一个是配置，一个是实时状态）。

`mount` 命令常用用法对照表

命令	作用 (日本語)	关键点
mount	マウント一覧を表示	查看当前实时的挂载状态
mount -a	一括マウント	按照 /etc/fstab 的设定进行挂载
mount /dev/sdb1 /mnt	手動マウント	将特定设备挂载到指定目录

🧠 小学生级记忆法：“点名时间”

想象 `mount` 是班主任：

- 1. 不带参数 (`mount`)：班主任走进教室大喊一声：“现在谁在位置上？”（列出已挂载的设备）。
- 2. 带 `-a` 参数 (`mount -a`)：班主任拿着名单 (`/etc/fstab`) 大喊：“名单上的人都给我坐到位置上去！”（按配置挂载）。

所以，什么都不加，就是在做“现有人数统计”。

✅ 正确答案

現在マウントされているファイルシステムの一覧を表示

📝 考试小秘籍

- 查看来源：在现代 Linux 中，`mount` 命令显示的信息主要来源于 `/proc/self/mounts` 或 `/etc/mtab`。
- 类似命令：`df` 命令也可以查看挂载情况，但它侧重于显示磁盘的“剩余空间”；而 `mount` 侧重于显示“挂载参数”。
- 只读挂载：如果只想看特定类型（如 ext4）的挂载，可以使用 `mount -t ext4`。

这一知识点主要考察 Linux 中「スワップ領域 (交换空间)」的管理命令。当系统内存不足时，Linux 会使用硬盘上的特定分区或文件作为虚拟内存，这个过程的“开关”控制是考试的常考项。

💡 知识点总结

管理スワップ（Swap）有一套专门的工具链：

- 作成 (建立)：使用 `mkswap` 将分区格式化为交换空间。
- 有効化 (启用)：使用 `swapon` 命令。
- 無効化 (禁用)：使用 `swapoff` 命令。

选项解析与详细说明

- ❌ `mkfs` :
 - 理由：用于创建普通的文件系统（如 ext4, xfs）。它不能用来处理交换空间。
- ❌ `swapon` :
 - 理由：这是“启动”开关。用于开启指定的分区或文件作为交换空间。
- ❌ `fdisk` :
 - 理由：磁盘分区工具。虽然可以用它创建分区并修改分区类型 ID（Swp 的 ID 通常是 82），但它不负责激活或禁用交换功能。
- ✅ `swapoff` : 正确答案。
 - 理由：专门用于「スワップを無効にする」（禁用交换空间）。执行后，系统会将交换空间中的数据移回物理内存并停止使用该空间。
- ❌ `mkswap` :
 - 理由：用于「スワップ領域の作成」（建立交换格式）。它相当于给分区打上“交换空间”的标签，但并不负责运行开关。

スワップ管理コマンド流程表

步骤	命令	作用 (日本語)	核心功能
1. 准备	<code>fdisk</code>	パーティション作成	划分出一块物理磁盘空间
2. 格式化	<code>mkswap</code>	スワップ署名の作成	初始化交换空间格式
3. 开启	<code>swapon</code>	スワップの有効化	开始使用虚拟内存
4. 关闭	<code>swapoff</code>	スワップの無効化	停止使用虚拟内存

小学生级记忆法：“电源开关”

想象 Swap 是一个「备用电池」：

- `mkswap` : 就是去商店买了一个空的备用电池。
- `swapon` : 把电池插上，「ON」开启电源。

3. `swapoff`：把电池拔掉，「OFF」关闭电源。

题目问的是“无效化”，所以直接找那个带「OFF」的按钮！

✓ 正确答案

`swapoff`

✎ 考试小秘籍

- **全关命令**：如果要一次性关闭所有已启用的交换空间，可以使用 `swapoff -a`。
- **查看状态**：除了 `swapon -s`，还可以通过查看 `/proc/swaps` 文件来确认当前的交换空间状态。
- **永久生效**：和挂载硬盘一样，如果希望开机自动启用 Swap，必须将其写入 `/etc/fstab` 文件。

这一知识点主要考察 **拡張正規表現 (Extended Regular Expressions, ERE)** 中的 **量指定子 (量词)** `+` 的用法。

💡 知识点总结

- `grep -E`：表示使用 **拡張正規表現** 进行匹配。
- `u+`：这里的 `+` 是量词，表示前面的字符（即 `u`）必须「1回以上の繰り返し」（出现 1 次或多次）。
- `bu+!` 的含义：匹配以 `b` 开头，中间跟着 **至少一个** `u`，最后以 `!` 结尾的字符串。

🔍 选项解析与详细说明

我们来逐行对比 `test.txt` 的内容：

- **✗ b!**
 - **理由**：`b` 和 `!` 之间没有 `u`。由于 `+` 要求至少有 1 个 `u`，所以不匹配。
- **✗ ba!**
 - **理由**：中间是 `a` 而不是 `u`，字符不匹配。
- **✓ bu!**
 - **理由**：包含 1 个 `u`，满足 `u+`（1 次以上）的条件。
- **✓ buu!**
 - **理由**：包含 2 个 `u`，满足 `u+`。

-  **buuu!**
 - **理由**：包含 3 个 `u`，满足 `u+`。

 正则表达式量词排查表

符号	含义 (日本語)	bu?! (匹配示例)	bu*! (匹配示例)	bu+! (匹配示例)
?	0次或1次	b!, bu!	-	-
*	0次以上	-	b!, bu!, buu!	-
+	1次以上	-	-	bu!, buu!, buuu!

 小学生级记忆法：“至少要有一个”

想象 `+` 是一个「贪吃鬼」：

1. 看到 `bu+!`，贪吃鬼说：“我必须吃到 **至少一个** `u` 才能干活！”
2. `b!`：没有 `u` 给它吃，它罢工了（不匹配）。
3. `bu!`、`buu!`：有 `u` 吃，不管是一个还是十个，它都高兴地干活（匹配）。

记住：`+` 就是“加号”，意味着必须有东西加进来（至少 1 个）。

 正确答案

- **bu!**
- **buu!**
- **buuu!**

 考试小秘籍

- **区分 `-E`**：普通 `grep` 不支持 `+`，必须加 `-E` 或者写成 `\+`。
- **区分 `*` 和 `+`**：这是最经典的考点。`*` 包含 **0次**（即 `b!` 也能匹配），而 `+` **不包含 0次**。
- **锚点符号**：如果正则写成 `^bu+!$`，则表示整行必须完全符合这个格式；如果没加 `^` 和 `$`，则只要行内包含这个片段就会被搜出来。

这一知识点主要考察 **拡張正規表現 (Extended Regular Expressions, ERE)** 中的 **量指定子 (量词)** `?` 的用法。

💡 知识点总结

- `grep -E`：表示启用扩展正则表达式模式。
- `?`：表示前面的字符（即 `u`）出现「0回または1回」（0次或1次）。
- `bu?!` 的含义：匹配以 `b` 开头，中间跟着 0 个或 1 个 `u`，最后以 `!` 结尾的字符串。

🔍 选项解析与详细说明

我们来逐行对比 `test.txt` 的内容：

- ☒ `b!`
 - 理由：`b` 和 `!` 之间有 0 个 `u`。由于 `?` 包含 0 次的情况，因此匹配。
- ☒ `ba!`
 - 理由：虽然也是 1 个字符，但它是 `a` 而不是 `u`。字符不匹配。
- ☒ `bu!`
 - 理由：`b` 和 `!` 之间有 1 个 `u`。由于 `?` 包含 1 次的情况，因此匹配。
- ☒ `buu!`
 - 理由：包含 2 个 `u`。这超过了 `?` 规定的“最多 1 个”的限制。
- ☒ `buuu!`
 - 理由：包含 3 个 `u`。不符合 0 次或 1 次的规则。

📊 正则表达式量词对比表

符号	含义 (日本語)	匹配次数	bu...! 匹配示例
<code>?</code>	0回または1回	0, 1	<code>b!</code> , <code>bu!</code>
<code>+</code>	1回以上	1, 2, 3...	<code>bu!</code> , <code>buu!</code> , <code>buuu!</code>
<code>*</code>	0回以上	0, 1, 2...	<code>b!</code> , <code>bu!</code> , <code>buu!</code>

🧠 小学生级记忆法：“有没有都行，但不能多”

想象 `?` 是一个「挑剔的快递员」：

1. 看到 `bu?!` ，快递员说：“我可以送 没有 `u` 的包裹（`b!`），也可以送 只有 1 个 `u` 的包裹（`bu!`）。”

2. `buu!`：他看到有两个 `u`，觉得太重了，于是拒绝配送（不匹配）。

3. 结论：`?` 的意思就是“最多一个”。

✅ 正确答案

- `b!`
- `bu!`

📝 考试小秘籍

- `-E` 的重要性：普通的 `grep` 不识别 `?`。必须使用 `grep -E` 或者 `egrep` 才能让 `?` 起作用。
- 常见陷阱：很多人会漏掉 `b!`。记住，`?` 包含了“完全没有”的情况。
- 进阶考点：如果题目给的是 `bu{1,2}!`，那意思就是必须有 1 到 2 个 `u`（即匹配 `bu!` 和 `buu!`）。

这一知识点主要考察 `sed` コマンド 与 正規表現 (正则表达式) 的配合使用，特别是如何精确匹配特定的字符串。

💡 知识点总结

- `sed` 命令：流编辑器，常用于文本的 置換 (替换)。格式为 `'s/旧字符串/新字符串/g'`。
- `.` (点号)：匹配 任意 1 个字符。
- `*` (星号)：匹配前面的字符 0 次或多次。
- `.*` (点星)：匹配 任意长度的任意字符（贪婪匹配）。

🔍 选项解析与详细说明

目标是将 `ping-t` 转换为 `HOGE`。

- ❌ `sed -e 's/p*t/HOGE/g' test.txt`
 - 理由：`p*` 表示 0 个或多个 `p`。它会匹配 `t`、`pt`、`ppt` 等，无法匹配中间的 `ing-`。
- ❌ `sed -e 's/p.*t/HOGE/g' test.txt`
 - 理由：`.*` 是贪婪匹配，它会匹配从第一个 `p` 到最后一个 `t` 之间的所有内容。如果一行中有多个 `t`，它会把中间的一大串全部换掉，不够精确。
- ❌ `sed -e 's/ping/HOGE/g' test.txt`

- **理由：**只替换了 `ping`，结果会变成 `H0GE-t`，不符合题目要求的 `H0GE`。
-  `sed -e 's/p....t/H0GE/g' test.txt`：正确答案。
 - **理由：**
 - i. `p` 匹配开头的 `p`。
 - ii. 中间的 **四个点** `....` 精确匹配了 `i`、`n`、`g`、`-` 这四个字符。
 - iii. 最后的 `t` 匹配结尾。
 - iv. 这种写法正好精确覆盖了 `ping-t` 这 6 个字符。
-  `sed -e 's/[ping-t]/H0GE/g' test.txt`
 - **理由：**`[]` 表示 **字符类**，它会匹配括号里的 **任意一个字符**。结果会将所有的 `p`、`i`、`n`、`g`、`-`、`t` 分别替换成 `H0GE`，输出会变得一团糟。

正则表达式匹配逻辑表 (以 `ping-t` 为例)

正则模式	匹配结果示例	结果分析
<code>p...t</code>	<code>ping-t</code>	精确匹配 6 位字符
<code>p.*t</code>	<code>ping-t is...st</code>	匹配过长 (贪婪)
<code>p*t</code>	<code>pt, ppt</code>	无法匹配中间字符
<code>[ping-t]</code>	<code>p 或 i 或 n</code>	逐个字符匹配

小学生级记忆法：“萝卜蹲与填空题”

想象 `ping-t` 是 6 个要填的格子：`[p][i][n][g][-][t]`。

1. **点号** `.` 就是一个“**万能萝卜**”，一个萝卜占一个坑。
2. `p....t` 的意思就是：第一个坑必须是 `p`，最后那个坑必须是 `t`，中间无论是什么，**必须正好有 4 个萝卜**。
3. `ping-t` 中间的 `ing-` 正好是 4 个字符，萝卜坑填得满满当当，完美匹配！

正确答案

`sed -e 's/p....t/H0GE/g' test.txt`

考试小秘籍

- **区分大小写**：注意 `test.txt` 第二行是全大写的 `PING-T`。默认的 `sed` 是 **区分大小写** 的，所以只有第一行的小写会被替换。
- **g 的含义**：末尾的 `g` 代表 **global**。如果不加 `g`，每行只会替换第一个匹配到的目标。
- **转义符号**：如果想匹配真正的点号 `.`，需要写成 `\.`。

这一知识点主要考察 `fgrep` コマンド 的核心特性——它 **不支持正则表达式 (正規表現)**，而仅仅是将输入作为 **固定字符串 (Fixed string)** 进行匹配。

知识点总结

- **fgrep**：全称是 Fixed-string grep。它不识别 `|`、`*`、`.` 等特殊符号，所有符号都被视为普通文字。
- **| (管道符)**：在扩展正则表达式 (`grep -E`) 中表示 **「または (或者)」**。但在 `fgrep` 中，它只会被当作一个普通的 **竖线符号**。
- **匹配逻辑**：`fgrep 'ping-t|PING-T'` 意味着系统在寻找一个字面上长成 `ping-t|PING-T` (包含中间那根竖线) 的完整字符串。

选项解析与详细说明

我们来看 `test.txt` 的实际内容：

1. `ping-t is synthesis...` (不含 `|`)
 2. `PING-T keeps always...` (不含 `|`)
- **✗ PING-Tを含む行が出力される**：由于使用了 `fgrep`，它不会把 `|` 理解为“或”，所以不会单独匹配 `PING-T`。
 - **✗ ping-tとPING-Tが含まれる行が出力される**：这是 `grep -E 'ping-t|PING-T'` 的运行结果，不是 `fgrep` 的。
 - **✗ ping-tとPING-Tが含まれない行が出力される**：这是使用 `-v` (取反) 选项时的结果。
 - **✗ ping-tを含む行が出力される**：理由同第一项，`|` 阻碍了单纯的字符串匹配。
 - **✓ 何も出力されない：正确答案。**
 - **理由**：因为文件中没有任何一行字面上包含 `ping-t|PING-T` 这个带竖线的组合。`fgrep` 此时表现得非常“死板”，它在找那根竖线，但文件里没有。

`grep` 家族功能对比表

命令	等价命令	是否支持正则	ping-t PING-T 的匹配行为
grep	(基本型)	基本正则	匹配包含 ping-t PING-T 的行 (把 当普通字符)
egrep	grep -E	扩展正则	匹配 ping-t 或 PING-T
fgrep	grep -F	不支持 (固定字符串)	仅匹配包含这串完整字符 (含竖线) 的行

小学生级记忆法：“死脑筋的复印机”

想象 `fgrep` 是一台「死脑筋的复印机」：

- 你给它一张纸写着 `A|B`。
- 普通 `grep` 会动脑子想：“哦，你是要 A 或者 B 啊。”
- `fgrep` 则会瞪大眼睛找：“哪里有 `A` 后面紧跟着一根 `|` 再跟着一个 `B`？没找到？那我不印了！”

记住：`fgrep` 的 `F` 就是 Fixed（固定/死板），它不玩任何花样。

✅ 正确答案

何も出力されない

📝 考试小秘籍

- 速度优势：**虽然 `fgrep` 很死板，因为它不需要解析复杂的正则逻辑，所以在搜索超大文本中的纯字符串时，**速度是最快的**。
- 改写建议：**如果想让题目生效，应该改用 `grep -E` 或者 `egrep`。
- 特殊字符搜索：**如果你真的要搜文件里的星号 `*` 或问号 `?`，用 `fgrep` 反而最方便，因为你不需要给这些符号加转义斜杠 `\`。

这一知识点主要考察 拡張正規表現 (Extended Regular Expressions, ERE) 中的「または (或者)」符号 `|` 的用法。

💡 知识点总结

- `egrep` コマンド：等同于 `grep -E`，它支持扩展正则表达式，无需对特殊符号加反斜杠。
- `|` (パイプ符号)：在正则中表示「OR (或者)」，用于匹配多个候选字符串中的任意一个。
- 引用符 (引号)：在 Shell 中执行包含特殊字符（如 `|`）的命令时，建议使用单引号 `' '` 包裹正则表达式，以防止 Shell 将 `|` 解释为系统管道。

🔍 选项解析与详细说明

目标是从文件中同时提取出 `taro` 和 `hanako`。

- ❌ `egrep 'taro-hanako' test.txt`
 - 理由：这会寻找字面上包含 `taro-hanako`（带连字符）的一行。
- ❌ `egrep 'taro OR hanako' test.txt`
 - 理由：正则中没有 `OR` 这个关键字，它会去寻找字面上包含单词 "OR" 的行。
- ❌ `egrep '[taro|hanako]' test.txt`
 - 理由：`[]` 是 文字クラス (字符类)。它会匹配括号内的 任意一个字符（即匹配 `t` 或 `a` 或 `r` 或 `o` 或 `|` ...）。这会导致输出大量不相关的行。
- ❌ `egrep taro|hanako test.txt`
 - 理由：由于没有引号，Shell 会先解释 `|`。它会尝试运行 `egrep taro` 并将其输出通过管道传给一个不存在的命令 `hanako`，导致报错。
- ✅ `egrep 'taro|hanako' test.txt`：正确答案。
 - 理由：使用单引号保护了正则表达式，`|` 被正确识别为“或者”。它会匹配包含 `taro` 或 `hanako` 的行。

📊 `grep` 与 `egrep` 的“或者”语法对比

工具	正则表达式写法	说明
<code>grep</code> (基本型)	<code>'taro hanako'</code>	必须加反斜杠转义 <code>`</code>
<code>egrep</code> (扩展型)	<code>'taro hanako'</code>	**直接使用 <code>`</code>
<code>fgrep</code> (固定型)	(不支持)	会去搜字面上的竖线符号

🧠 小学生级记忆法：“点菜的选择权”

想象你在餐厅点餐：

1. **| 符号**：就像菜单上的“或”。`'taro|hanako'` 的意思就是：“服务员，给我来一份 `taro` 或者一份 `hanako`。”
2. **引号 ' '**：就像是“菜单本”。如果你不把菜名写在菜单本里直接喊（不加引号），服务员（Shell）可能会听岔了，以为你要他去干别的活（系统管道）。

记住：选 A 或 B，中间插根“棍子”（`|`）。

✅ 正确答案

`egrep 'taro|hanako' test.txt`

📝 考试小秘籍

- **等价命令**：记住 `egrep` 完全等同于 `grep -E`。如果选项里没 `egrep` 但有 `grep -E`，选它准没错。
- **字符类陷阱**：千万别把 `|` 放进 `[]` 里。在 `[]` 里，所有符号（除了极个别）都只是普通字符。
- **多选一**：你可以连接多个 `|`，比如 `'A|B|C|D'`，这样四个中只要中一个就会被搜出来。

这一知识点主要考察 `fgrep` コマンド 的核心特性——它 **不支持正则表达式 (正規表現)**，而仅仅是将输入作为 **固定字符串 (Fixed string)** 进行匹配。

💡 知识点总结

- **fgrep**：全称是 Fixed-string grep。它不识别 `|`、`*`、`.` 等特殊符号，所有符号都被视为普通文字。
- **| (管道符)**：在扩展正则表达式 (`grep -E`) 中表示「または (或者)」。但在 `fgrep` 中，它只会被当作一个普通的 **竖线符号**。
- **匹配逻辑**：`fgrep 'ping-t|PING-T'` 意味着系统在寻找一个字面上长成 `ping-t|PING-T`（包含中间那根竖线）的完整字符串。

🔍 选项解析与详细说明

我们来看 `test.txt` 的实际内容：

1. `ping-t is synthesis...`（不含 `|`）

2. `PING-T keeps always...` (不含 `|`)
- ✗ `PING-T`を含む行が出力される: 由于使用了 `fgrep`，它不会把 `|` 理解为“或”，所以不会单独匹配 `PING-T`。
 - ✗ `ping-t`と`PING-T`が含まれる行が出力される: 这是 `grep -E 'ping-t|PING-T'` 的运行结果，不是 `fgrep` 的。
 - ✗ `ping-t`と`PING-T`が含まれない行が出力される: 这是使用 `-v` (取反) 选项时的结果。
 - ✗ `ping-t`を含む行が出力される: 理由同第一项，`|` 阻碍了单纯的字符串匹配。
 - ✓ **何も出力されない: 正确答案。**
 - 理由: 因为文件中没有任何一行字面上包含 `ping-t|PING-T` 这个带竖线的组合。`fgrep` 此时表现得非常“死板”，它在找那根竖线，但文件里没有。

 `grep` 家族功能对比表

命令	等价命令	是否支持正则	<code>ping-t PING-T</code> 的匹配行为
<code>grep</code>	(基本型)	基本正则	匹配包含 <code>ping-t PING-T</code> 的行 (把 <code> </code> 当普通字符)
<code>egrep</code>	<code>grep -E</code>	扩展正则	匹配 <code>ping-t</code> 或 <code>PING-T</code>
<code>fgrep</code>	<code>grep -F</code>	不支持 (固定字符串)	仅匹配包含这串完整字符 (含竖线) 的行

 小学生级记忆法：“死脑筋的复印机”

- 想象 `fgrep` 是一台「死脑筋的复印机」：
- 你给它一张纸写着 `A|B`。
 - 普通 `grep` 会动脑子想：“哦，你是要 A 或者 B 啊。”
 - `fgrep` 则会瞪大眼睛找：“哪里有 `A` 后面紧跟着一根 `|` 再跟着一个 `B`？没找到？那我不印了！”
- 记住: `fgrep` 的 `F` 就是 Fixed (固定/死板)，它不玩任何花样。

✓ 正确答案

考试小秘籍

- **速度优势**：虽然 `fgrep` 很死板，但因为它不需要解析复杂的正则逻辑，所以在搜索超大文本中的纯字符串时，**速度是最快的**。
 - **改写建议**：如果想让题目生效，应该改用 `grep -E` 或者 `egrep`。
 - **特殊字符搜索**：如果你真的要搜文件里的星号 `*` 或问号 `?`，用 `fgrep` 反而最方便，因为你不需要给这些符号加转义斜杠 `\`。
-

grep コマンドを使用し、以下の「test.txt」ファイルのうち大文字アルファベットを含まない行を表示しなさい。

```
$ cat test.txt
```

NAME

grep, egrep, fgrep - print lines matching a pattern

SYNOPSIS

```
grep [options] PATTERN [FILE...]
```

```
grep [options] [-e PATTERN | -f FILE] [FILE...]
```

这道题目考察的是 `grep` コマンド 的「除外 (排除)」逻辑，以及 正規表現 (正则表达式) 中 文字クラス (字符类) 的写法。

知识点总结

- `[A-Z]`：匹配任何一个 **大写字母**。
 - `-v` オプション：执行 **反転検索 (反向搜索)**。它会显示「一致しない行」(不匹配的行)。
 - `[^...]`：在字符类内部，`^` 表示「否定」，匹配不在括号内的字符。
-

选项解析与详细说明

目标是：显示 **不包含** 大写字母的行。

-  `grep -i '[a-z]' test.txt`
 - **理由**：`-i` 表示 **忽略大小写**。这会匹配所有含有字母（不论大小）的行。
-  `grep -v '[A-Z]' test.txt`：正确答案。

- 理由：
 - i. `[A-Z]` 匹配所有含有大写字母的行。
 - ii. `-v` 将结果反转，即排除掉这些含有大写字母的行。
 - iii. 最终留下的就是“完全不含大写字母”的行。
- ✗ `grep '[^A-Z]' test.txt`
 - 理由：这表示匹配“包含至少一个非大写字母字符”的行。只要行里有一个小写字母、空格或标点，该行就会被抓出来，即便它同时也含有大写字母。
- ✗ `grep '^[^A-Z]' test.txt`
 - 理由：匹配“以非大写字母开头”的行。
- ✗ `grep '^[^a-z]' test.txt`
 - 理由：匹配“不以小写字母开头”的行。
- ✗ `grep '[a-z]' test.txt`
 - 理由：匹配“含有小写字母”的行。

grep 排除逻辑对比表

需求	命令写法	逻辑说明
排除包含 A 的行	<code>grep -v 'A'</code>	整行消失 (只要行内有 A 就不显示)
匹配非 A 的字符	<code>grep '[^A]'</code>	字符过滤 (只要行内有一个字符不是 A，该行就显示)

小学生级记忆法：“全家连坐法”

想象每一行是一个「小家庭」：

- `grep '[A-Z]'`：警察说：“把家里有 **大个子 (大写字母)** 的家庭都找出来。”
- `-v` 开关：警报反转。警察改口说：“把刚才那些有大个子的家庭全关起来，**只准没大个子的家庭出来！**”
- 结果：只有清一色小矮人（小写字母）或没人的家庭（空行、符号行）能通过。

记住：要排除“包含某物”的整行，必须用 `-v`。

正确答案

`grep -v '[A-Z]' test.txt`

考试小秘籍

- **-v 的威力**：这是 Linux 运维中最常用的过滤手段。比如 `grep -v '^#'` 可以排除所有以 `#` 开头的注释行。
- **字符类 `[^]` 的陷阱**：它是针对 **单个字符** 的否定，而不是针对整行的排除。这是初学者最容易混淆的地方。
- **空行处理**：`grep -v '[A-Z]'` 也会显示纯粹的空行，因为空行里确实“不包含大写字母”。

以下の「test.txt」ファイルのうち、「taro」の行にのみマッチするパターンを選びなさい。

```
$ cat test.txt
taro
tarosuke
taichi
kisuke
```

这道题目考察的是 **正規表現 (正则表达式)** 中 **アンカー (锚点)** 的用法，特别是如何实现 **完全一致 (完全匹配)**。

知识点总结

在 Linux 的 `grep` 或其他工具中，要精确匹配一行内容，需要使用边界定位符：

- `^`：匹配行首 (行の先頭)。
- `.`：匹配任意一个字符 (任意の1文字)。
- `$`：匹配行尾 (行の末尾)。
- **完全匹配公式**：使用 `^内容$` 可以确保整行内容与模式完全一致，没有任何多余字符。

选项解析与详细说明

目标是从文件中仅匹配「taro」这一行。

-  `^t..o\$`：
 - **理由**：在正则表达式中，`$` 本身就是行尾锚点。如果加了反斜杠 `\$`，它就会变成匹配字面意义上的“美元符号”，而不是匹配行尾。
-  `^t*`：
 - **理由**：`*` 表示前面的字符重复 0 次以上。这个模式会匹配所有以 0 个或多个 `t` 开头的行。这会导致匹配到文件中所有的行（包括 `kisuke`）。

- ✗ `[taro]` :
 - 理由：这是字符类，匹配 `t`、`a`、`r`、`o` 中的 **任意一个** 字符。它会匹配到文件里的所有行。
- ✗ `...o` :
 - 理由：匹配任何以 3 个任意字符开头并以 `o` 结尾的片段。它不仅会匹配 `taro`，还会匹配 `tarosuke` 中的前四个字符部分。
- ✓ `^t..o$` : 正确答案。
 - 理由：
 - i. `^t`：必须以 `t` 开头。
 - ii. `..`：中间必须正好有两个任意字符（匹配 `ar`）。
 - iii. `o$`：必须以 `o` 结尾。
 - iv. 由于开头和结尾都锁死了，它不会匹配到更长的 `tarosuke`，也不会匹配到 `taichi`。

📊 匹配结果对比表 (针对 test.txt)

正则模式	匹配行 (taro)	匹配行 (tarosuke)	匹配行 (taichi)	匹配行 (kisuke)
<code>^t..o\$</code>	✓	✗	✗	✗
<code>...o</code>	✓	✓	✗	✗
<code>^t*</code>	✓	✓	✓	✓
<code>[taro]</code>	✓	✓	✓	✗

🧠 小学生级记忆法：“两头堵死”

想象每一行是一个「长廊」：

- `^` (大门)：在长廊最左边装个大门。
- `$` (后门)：在长廊最右边装个后门。
- `^t..o$` 的意思就是：从大门进来第一个人必须是 `t`，中间只能站 2 个人，最后一个人必须是 `o` 且直接就是后门。
- `tarosuke` 因为太长了，`o` 后面还有人，够不到后门，所以被关在外面了。

✓ 正确答案

📝 考试小秘籍

- **转义陷阱**：记住在 `grep` 的基本正则中，`$` 是特殊符号。只有当你真的要搜内容里的 `$` 时才加 `\$`。
- **点号数量**：`.` 的个数必须和字符个数严格对应。
- **其他工具**：如果是 `grep -x "taro"`，也能实现这种全行匹配的效果，不需要写锚点。

grepコマンドの書式と主なオプションは以下のとおりです。

grep [オプション] 検索パターン [ファイル名]

オプション	説明
-c	マッチした行の行数のみ表示
-f	検索パターンをファイルから読み込む
-i	大文字と小文字を区別しない
-n	先頭に行番号をつけて、マッチした行を表示
-v	マッチしなかった行を表示
-E	拡張正規表現を使用(egrepコマンドと同様)
-F	検索パターンを正規表現ではなく、固定文字列とする(fgrepコマンドと同様)

这道题目考察的是 **拡張正規表現 (Extended Regular Expressions, ERE)** 中用于表示「または (或者)」的逻辑运算符。

💡 知识点总结

在正则表达式中，如果我们想要匹配多个选项中的任意一个，需要使用「**選択 (选择)**」算符：

- **| (垂直バー)**：表示「**OR (或者)**」。它将前后的模式分开，匹配其中任何一个。
- **使用环境**：通常在 `grep -E` (或 `egrep`) 以及 `sed -r` 中直接使用。

🔍 选项解析与详细说明

- **✗ abc?xyz**：

- **理由：** `?` 是量词，表示前面的字符出现 **0次或1次**。这里的意思是匹配 `ab` 后面跟着 0 或 1 个 `c`，再跟着 `xyz`。
- **✗ `abc+xyz`:**
 - **理由：** `+` 表示前面的字符出现 **1次以上**。这里匹配 `abcxyz`、`abccxyz` 等。
- **✗ `\abc\xyz`:**
 - **理由：** 这不符合正则表达式的语法规则。反斜杠 `\` 通常用于转义特殊字符。
- **✓ `abc|xyz`: 正确答案。**
 - **理由：** `|` 正确表示了「**abc または xyz**」的逻辑。它会匹配包含字符串 `abc` 的行，或者包含 `xyz` 的行。
- **✗ `abc*xyz`:**
 - **理由：** `*` 表示前面的字符出现 **0次以上**。

 常见正则符号功能排查表

符号	含义 (日本語)	逻辑功能	示例匹配 (abc...xyz)
<code>**`</code>	<code>`**</code>	選択 (OR)	匹配 A 或 B
<code>.</code>	任意の1文字	匹配任意一个占位符	<code>abdxyz</code> , <code>ab1xyz</code>
<code>*</code>	0回以上の繰り返し	匹配 0 到无数个	<code>abxyz</code> , <code>abccxyz</code>
<code>+</code>	1回以上の繰り返し	匹配 1 到无数个	<code>abcxyz</code> , <code>abccxyz</code>
<code>?</code>	0回または1回	最多匹配 1 个	<code>abxyz</code> , <code>abcxyz</code>

 小学生级记忆法：“独木桥的选择”

想象 `|` 是一根立在中间的「分界杆」：

1. 路口有一个告示牌写着 `abc|xyz`。
2. 这根杆子把路分成了左边 (`abc`) 和右边 (`xyz`)。
3. 你可以选择从 **左边走 (abc)**，也可以选择从 **右边走 (xyz)**。
4. 不管走哪边，只要过了这根杆子就算匹配成功！

记住：看到“或者”就找那根“棍子” (`|`)。

✅ 正确答案

abc|xyz

📝 考试小秘籍

- **括号的使用**：如果你想匹配 `ab(c|x)yz`，意思就是匹配 `abcyz` 或者 `abxyz`。括号可以限制 `|` 的作用范围。
- **工具区别**：普通的 `grep` 不支持 `|`，必须加反斜杠写成 `\|`，或者直接用 `grep -E`。
- **连用**：`|` 可以连续使用，比如 `A|B|C` 表示 A、B、C 三选一。

这道题目考察的是 Linux 中 **man 手册 (在线帮助手册)** 相关数据库的维护工具。为了能让 `whatis` 或 `man -f` 快速检索到命令的简短描述，系统需要预先生成一个索引数据库。

💡 知识点总结

- **whatis 数据库**：包含了所有命令名称及其简短描述的索引文件。
- **检索命令**：使用 `whatis` 或 `man -f` 进行查询。
- **数据库路径**：通常存放在 `/var/cache/man/whatis` 或各 `man` 路径下。
- **生成/更新工具**：由于手册页会随软件安装而增加，系统需要定期扫描并重建索引。

🔍 选项解析与详细说明

- ❌ **dbwhatis / updatewhatis / whatisdb / makewhisdb**：
 - **理由**：这些都不是标准的 Linux 命令名称。虽然某些发行版可能有类似的脚本，但在 LPI 等标准考试中，它们属于干扰项。
- ✅ **makewhatis**：正确答案。
 - **理由**：`makewhatis` 是传统的用于扫描 `man` 页面并创建/更新 `whatis` 数据库的命令。
 - **注意**：在现代的 Linux 发行版（如 CentOS 7+, Ubuntu 等）中，该命令通常被 `mandb` 命令所取代，但在考试题库中，`makewhatis` 依然是代表性的标准答案。

📊 man 手册工具对照表

功能	命令 (日本語)	常用选项/等价命令
创建/更新数据库	<code>makewhatis</code>	<code>mandb</code> (现代系统常用)
完全匹配查询	<code>whatis</code>	<code>man -f</code>
部分匹配(关键词)查询	<code>apropos</code>	<code>man -k</code>

🧠 小学生级记忆法：“制作 (Make) 一个 ‘是什么’ (Whatis)”

想象你要给图书馆做一个索引卡片：

1. 你手里有很多书（`man` 页面）。
2. 你想知道每本书大概讲了什么（`whatis`）。
3. 所以你需要 “制作 (Make)” 一个 “是什么 (Whatis)” 的清单。
4. 拼在一起就是：`makewhatis`。

✅ 正确答案

`makewhatis`

📝 考试小秘籍

- **新旧更替**：如果选项里没有 `makewhatis` 却有 `mandb`，请选 `mandb`。它们的作用是一样的。
- **权限要求**：因为要往系统目录写数据库，执行 `makewhatis` 通常需要 **root 权限** (Section 8)。
- **关联命令**：记住 `man -f` 和 `whatis` 是亲兄弟（完全一致），`man -k` 和 `apropos` 是亲兄弟（部分一致）。

「/etc/fstab」 ファイルを編集して、ext3ファイルシステムの「/dev/sda1」をシステム起動時から「/boot」に自動マウントするよう設定したい。適切な記述は次のうちどれか。(全て選択)

💡 知识点总结

`/etc/fstab` 是 Linux 系统中最重要配置文件之一，它决定了系统启动时如何自动挂载硬盘分区。

每一行配置都必须严格遵守 **6 个字段** 的顺序，缺一不可，顺序错了系统可能无法启动。

🔍 选项解析与详细说明

题目要求：将 `ext3` 格式的 `/dev/sda1` 分区在开机时自动挂载到 `/boot` 目录。

- ✓ `/dev/sda1 /boot ext3 auto 1 1`：正确答案。
 - 理由：顺序正确（设备 → 挂载点）。选项中的 `auto` 明确表示“自动挂载”。
- ✗ `/boot /dev/sda1 ext3 auto 1 1`
 - 理由：顺序颠倒。必须先写“设备（源）”，再写“目录（目的地）”。
- ✗ `/dev/sda1 /boot ext3 noauto 1 1`
 - 理由：`noauto` 表示“不要自动挂载”。这与题目要求背道而驰。
- ✗ `/boot /dev/sda1 ext3 defaults 1 1`
 - 理由：同样是顺序颠倒。
- ✓ `/dev/sda1 /boot ext3 defaults 1 1`：正确答案。
 - 理由：`defaults` 是一个组合选项，它包含了 `rw`、`suid`、`dev`、`exec` 以及最重要的 `auto`。因此使用 `defaults` 也能实现自动挂载。

 `/etc/fstab` 字段排查表

字段顺序	含义 (中文)	内容示例	记忆要点
第 1 字段	设备名	<code>/dev/sda1</code>	“谁”要挂载？
第 2 字段	挂载点	<code>/boot</code>	挂载到“哪”？
第 3 字段	文件系统类型	<code>ext3</code>	是什么“格式”？
第 4 字段	挂载选项	<code>defaults</code>	有什么“规矩”？
第 5 字段	dump 备份	<code>1</code>	要不要“备份”？
第 6 字段	fsck 检查	<code>1</code>	开机要不要“扫描”？

 小学生级记忆法：“背着书包去学校”

你可以把 `fstab` 的一行想象成一个学生的行程：

1. `/dev/sda1`：我是**学生**（设备）。
2. `/boot`：我要去**学校**（挂载点）。
3. `ext3`：我穿的**校服**（格式）。
4. `defaults`：我遵守**校规**（选项，默认要准时到校 = `auto`）。

口诀：先有人（设备），再有地（挂载点），最后立规矩（选项）！

✅ 正确答案

- `/dev/sda1 /boot ext3 auto 1 1`
- `/dev/sda1 /boot ext3 defaults 1 1`

📝 考试小秘籍

- **defaults 的秘密**：LPI 经常考 `defaults` 包含哪些子项。记住：它包含 `auto`，但不包含 `noauto`。
- **测试命令**：修改完 `fstab` 后，千万不要直接重启！先运行 `mount -a`，如果没报错，说明配置正确，可以安全重启。
- **最后两个数字**：通常根分区 `/` 的最后两个数字是 `1 1`，其他分区（如 `/boot`）通常是 `1 2` 或 `0 0`。但在选择题中，只要前四个字段对，基本就是正确答案。

这道题目考察的是 Linux 系统中「リアルタイム (实时)」挂载信息的存储位置。在 Linux 中，很多系统状态并不存储在硬盘文件中，而是存在于内存中的 **虚拟文件系统** 里。

💡 知识点总结

- `/etc/fstab`：「設定ファイル (配置文件)」。记录的是“希望”系统开机时挂载哪些分区。
- `/proc/mounts`：「カーネル情報 (内核信息)」。记录的是当前系统“实际上”已经挂载的所有文件系统信息。
- `/etc/mtab`：也是记录当前挂载信息的文件，但在现代系统中，它通常只是 `/proc/self/mounts` 的一个符号链接。

🔍 选项解析与详细说明

- ✅ `/proc/mounts`：正确答案。
 - **理由**：`/proc` 是一个虚拟文件系统，反映了内核的实时状态。`/proc/mounts` 会实时列出当前所有已成功挂载的设备、挂载点及参数。
- ❌ `/etc/fstab`
 - **理由**：这是静态的配置文件。如果一个分区写在里面但挂载失败了，或者你手动挂载了一个没写进去的分区，`fstab` 都无法反映真实情况。
- ❌ `/proc/mount/tab`
 - **理由**：这是干扰项，Linux 中不存在这个路径。
- ❌ `/etc/self`

- 理由：这也是干扰项。虽然 `/proc/self` 存在（代表当前进程），但 `/etc/self` 是没有意义的。

 挂载信息“虚实”对照表

文件路径	类型	作用 (日本語)	特点
<code>/etc/fstab</code>	静止状态	設定ファイル	记录“计划”挂载的内容
<code>/proc/mounts</code>	运动状态	カーネルのリアルタイム情報	记录“目前”正在挂载的内容

 小学生级记忆法：“名单 vs 到场人数”

想象学校要开运动会：

- `/etc/fstab`：是「报名表」。上面写着谁应该参加（开机应该挂载谁）。
- `/proc/mounts`：是「点名册」。老师现在去操场上数人，看看到底谁真的站在那儿（当前挂载了谁）。

题目问的是“现在挂载的信息”，所以要看“点名册” (`/proc/mounts`)!

 正确答案

`/proc/mounts`

 考试小秘籍

- 三位一体：记住 `mount` 命令（不带参数）、`cat /proc/mounts` 和 `cat /etc/mtab` 这三个操作看到的内容几乎是一样的。
- `/proc` 必考点：只要看到题目问“实时状态”、“内核参数”、“硬件信息”，答案大概率在 `/proc` 目录下。
- 注意链接：在考试中如果出现 `/etc/mtab` 也是正确的，但通常 `/proc/mounts` 是更底层的标准答案。

这道题目考察的是当 `/etc/fstab` 文件中已经存在对应配置时，`mount` 命令的简化用法。

 知识点总结

- **/etc/fstab 的联动：**该文件是挂载的“预设名单”。如果设备和挂载点的对应关系已经写在里面，`mount` 命令可以实现“半自动”识别。
- **简化命令：**在有预设的情况下，你只需要提供“设备名”或“挂载点”其中之一，内核就会自动去 `fstab` 里找剩下的那个参数。
- **标准写法：**即便是有了预设，手动写出完整的“设备 + 挂载点”依然是有效的。

🔍 选项解析与详细说明

题目背景： `/dev/hdc`（源）和 `/mnt/cdrom`（目的地）的对应关系已在 `fstab` 中定义。

- ❌ `mount /mnt/cdrom /dev/hdc`
 - **理由：顺序反了。** `mount` 的标准格式是 `mount [设备] [挂载点]`。这个选项把目的地写在了前面。
- ❌ `mount`
 - **理由：**不带任何参数的 `mount` 是用来 **列出当前已挂载的信息**，而不是执行新的挂载操作。
- ✅ `mount -t iso9660 /dev/hdc /mnt/cdrom`：正确答案。
 - **理由：**这是最完整的标准写法。 `-t iso9660` 指定了 CD-ROM 常用的文件系统类型，后接设备名和挂载点。即便没有 `fstab` 预设，这一条也能成功。
- ✅ `mount /dev/hdc`：正确答案。
 - **理由：**由于 `fstab` 已有记录，命令会自动补全挂载点为 `/mnt/cdrom`。
- ✅ `mount /mnt/cdrom`：正确答案。
 - **理由：**同上，命令会自动从 `fstab` 中找回对应的设备 `/dev/hdc`。

📊 `mount` 命令参数匹配表

场景	命令示例	依赖条件
全手动 (最稳)	<code>mount /dev/hdc /mnt/cdrom</code>	无
半自动 (找目的地)	<code>mount /dev/hdc</code>	<code>/etc/fstab</code> 已定义
半自动 (找源设备)	<code>mount /mnt/cdrom</code>	<code>/etc/fstab</code> 已定义
全自动 (批量)	<code>mount -a</code>	挂载所有 <code>fstab</code> 中的设备

🧠 小学生级记忆法：“默契的舞伴”

想象 `/dev/hdc` 和 `/mnt/cdrom` 是一对已经在民政局领了证（写进 `fstab`）的「舞伴」：

- 1. 全名呼唤：你喊“张三（设备）和李四（挂载点）跳舞”，没问题。
- 2. 只喊一个：因为他们领过证了，你只要在大厅喊一声“张三出来跳舞！”，李四就会自动跑过来跟他搭档。
- 3. 结论：只要证办好了，喊谁的名字都能成对！

✅ 正确答案

- mount -t iso9660 /dev/hdc /mnt/cdrom
- mount /dev/hdc
- mount /mnt/cdrom

📝 考试小秘籍

- 陷阱：顺序：考试最喜欢把设备和挂载点位置调换来考你，记住永远是「从哪来 (Device) 到哪去 (Dir)」。
- iso9660：这是 CD-ROM 的标准文件系统格式，看到这个词基本就和光盘挂载有关。
- -a 选项：如果是 mount -a，它会尝试挂载 fstab 中所有 没有 noauto 标记 的行。

オプション	説明
async	非同期で入出力を行う
auto	「mount -a」コマンド実行時にマウント
noauto	「mount -a」コマンド実行時にマウントしない
defaults	デフォルト指定 (async, auto, dev, exec, nouser, rw, suid)
exec	バイナリの実行を許可
noexec	バイナリの実行を禁止
ro	読み取り専用
rw	読み書きを許可
suid	SUIDとSGIDを有効化
user	一般ユーザでマウント可、本人のみアンマウント可
users	一般ユーザでマウント可、誰でもアンマウント可
nouser	一般ユーザによるマウントを禁止

💡 知识点总结

在 /etc/fstab 的第 4 列（选项列）中，我们可以定义文件系统的行为。

- 读写控制：决定了该分区是否允许存储或修改数据。
- 默认行为：如果你在这一列写 defaults，系统会自动包含 rw（读写）等一系列常用权限。

选项解析与详细说明

目标是：找到允许「読み書き (读写)」的选项。

- ❌ **auto**
 - 理由：控制是否在开机或执行 `mount -a` 时“自动挂载”。它不决定你挂载上去后能不能写数据。
- ❌ **users**
 - 理由：允许“普通用户”执行挂载和卸载操作。默认情况下只有 root 能挂载。
- ❌ **ro**
 - 理由：Read-Only 的缩写。表示“只读”，禁止任何写入操作。常用于光盘或备份硬盘。
- ❌ **async**
 - 理由：表示“异步输入/输出”。数据先写到内存缓存，稍后再同步到磁盘，这能提高速度，但不控制读写权限。
- ✅ **rw**：正确答案。
 - 理由：Read-Write 的缩写。表示“读写”，允许用户查看并修改该分区上的文件。

读写权限选项对照表

选项	含义 (日本語)	写入权限	适用场景
rw	読み書き可能	允许	硬盘分区、U 盘
ro	読み込み専用	禁止	CD-ROM、系统保护分区

小学生级记忆法：“笔与橡皮 (Pen & Eraser)”

- ro**：Read Only（只读）。就像在博物馆看画，只能用眼睛 看 (Read)，不能动手画。
- rw**：Read & Write（读写）。就像在自己的笔记本上，既能 看 (Read)，也能用笔 写 (Write)。

口诀：看到 W (Write) 就能写，只有 O (Only) 只能看！

✅ 正确答案

rw

考试小秘籍

- **defaults 的坑**：考试常问 `defaults` 包含什么。记住它包含 `rw`，所以默认挂载的硬盘都是可以写的。
- **安全建议**：对于系统关键目录（如 `/boot`），有时为了安全会故意设为 `ro`。
- **重新挂载**：如果你发现硬盘只能读不能写，可以尝试在线修改模式：`mount -o remount,rw /挂载点`。

这一知识点主要考察 Linux 系统中用于「マウント (挂载)」与「アンマウント (卸载)」文件系统的基本命令。

知识点总结

- **mount**：用于将存储设备（如硬盘分区、光盘、U 盘）关联到文件系统树的某个目录（挂载点）。
- **umount**：用于解除设备与目录之间的关联。
- **虚拟文件系统**：挂载操作让 Linux 能够通过单一的目录树结构访问不同的物理设备。

选项解析与详细说明

-  **mkfs**
 - **理由**：全称是 **make kile system**。用于在磁盘分区上「作成 (创建)」文件系统（即格式化），而不是挂载。
-  **fsumount**
 - **理由**：这是一个不存在的干扰项。
-  **umount**
 - **理由**：虽然它是合法命令，但它的功能是「アンマウント (卸载)」已挂载的文件系统，而不是挂载。
-  **fsmount**
 - **理由**：这也是一个不存在的干扰项。
-  **mount**：正确答案。
 - **理由**：这是执行文件系统「マウント (挂载)」的核心命令。

文件系统常用命令对照表

命令	含义 (日本語)	主要功能
mount	マウント	挂载设备到目录
umount	アンマウント	卸载设备
mkfs	ファイルシステム作成	格式化分区
fsck	ファイルシステムチェック	检查并修复文件系统

 小学生级记忆法：“插座与电器”

- 1. `mount` (挂载): 就像把电器的 **插头插进插座**。插好后（挂载后），你才能通过插座（目录）使用电器（硬盘里的数据）。
- 2. `umount` (卸载): 就像 **拔掉插头**。拔掉后，你就不能再通过这个插座用电了。

口诀：要用数据先 `mount`（插上），不用数据要 `umount`（拔掉）！

 正确答案

`mount`

 考试小秘籍

- **拼写陷阱**：卸载命令是 `umount`，中间 **没有字母 n**（不是 unmount）。这是 LPI 考试中最经典的拼写坑。
- **查看挂载情况**：直接输入不带参数的 `mount` 命令，可以列出当前系统所有已挂载的设备。
- **权限要求**：执行挂载和卸载通常需要 **root** 权限。

mountコマンドの書式と主なオプションは以下のとおりです。

mount [オプション] [マウントするデバイス名] [マウントポイント]

オプション	説明
-a	「/etc/fstab」ファイルに記載されているファイルシステムをすべてマウント (noautoマウントオプションが指定されているものは除く)
-o	続けてマウントオプションを指定
-t	続けてファイルシステムの種類を指定 (指定しない場合は自動で種類を推測)

这一知识点主要考察 `mount` コマンド 的选项及其参数的「指定順序 (指定顺序)」。

💡 知识点总结

- `-t` オプション: 用于指定 ファイルシステムのタイプ (文件系统类型), 如 `ext3`, `xf``s`, `iso9660` 等。
- `-o` オプション: 用于指定 マウントオプション (挂载选项), 如 `ro` (只读), `rw` (读写), `remount` (重新挂载) 等。
- 参数顺序: 标准的挂载命令格式为 `mount [选项] <デバイス名 (设备)> <マウントポイント (目录)>`。

🔍 选项解析与详细说明

目标: 将 `ext3` 格式的 `/dev/sda1` 挂载到 `/export`, 且要求 只读 (Read-Only)。

-  `mount -t ext3 -o ro /dev/sda1 /export` : 正确答案。
 - 理由: 类型 (`-t ext3`) 和选项 (`-o ro`) 填写正确。最重要的是, 顺序是“设备”在前, “目录”在后。
-  `mount -t ro -o ext3 /export /dev/sda1`
 - 理由: 把 `ro` 当成了文件类型, 把 `ext3` 当成了选项, 且设备和目录的顺序反了。
-  `mount -t ext3 -o ro /export /dev/sda1`
 - 理由: 虽然选项对, 但「引数の順番 (参数顺序)」错了。挂载时必须先说“源设备”, 再说“目的地目录”。
-  `mount -a ext3 /dev/sda1 /export`
 - 理由: `-a` 表示挂载 `/etc/fstab` 中的 所有 设备, 不能手动指定具体的设备和目录。
-  `mount -t ro -o ext3 /dev/sda1 /export`
 - 理由: 同样将 `ro` 误作为文件系统类型。

处理“设备忙”问题的排查表

如果在挂载或卸载时遇到问题，通常按以下步骤排查：

现象	可能原因	解决工具
Unknown fs type	-t 指定的类型错误	lsblk -f (确认类型)
Mount point does not exist	挂载点目录未创建	mkdir /export
Device is busy	有进程正在访问该目录	fuser -v 或 lsof

小学生级记忆法：“搬家逻辑”

想象你要把家具（数据）从旧房子（设备）搬到新房子（目录）：

1. `-t` (Type)：告诉搬家公司，家具是“木头的” (ext3)。
2. `-o` (Option)：要求搬家公司，只能“看不能摸” (ro)。
3. 顺序：你要先指着“旧房子” (`/dev/sda1`)，再说搬到“新房子” (`/export`)。

口诀：类型选项在前边，设备目录按顺序，从左往右去搬家！

正确答案

```
mount -t ext3 -o ro /dev/sda1 /export
```

考试小秘籍

- `-o ro` vs `rw`：这是最常考的两个选项对比。 `ro` 是只读， `rw` 是默认的读写。
- 省略 `-t`：在现代 Linux 中， `mount` 通常能自动识别文件系统类型，但在 LPI 考试中，显式写出 `-t` 是标准做法。
- `umount` 拼写：再次提醒，卸载命令是 `umount` （没有 n）。

umountコマンドの書式と主なオプションは以下のとおりです。

```
umount [オプション] [マウントされているデバイス名、またはマウントポイント]
```

オプション	説明
-a	「/etc/mtab」ファイルに記載されているファイルシステムをすべてアンマウント
-t	指定した種類のファイルシステムのみをアンマウント

この問題は、一括で **アンマウント (卸载)** を行う際の `umount` コマンド のオプションに関するものです。

 知识点总结

- `umount`：用于卸载已挂载的文件系统。
- `/etc/mtab`：记录当前系统中所有已挂载文件系统的文件（在现代系统中通常指向 `/proc/self/mounts`）。
- **一括操作**：当需要一次性卸载多个分区（例如关机前或维护时），可以使用特定选项来参照 `mtab` 或 `fstab` 执行批量操作。

 选项解析与详细说明

目标：找到能卸载 `/etc/mtab` 中所有文件系统的 `umount` 选项。

-  **-a：正确答案。**
 - **理由**：`-a` (all) 选项指示 `umount` 命令卸载 `/etc/mtab` 文件中列出的 **所有** 文件系统。在执行时，它通常会排除掉像 `/` (根分区) 这样正在被系统使用的核心分区。
-  **-A**
 - **理由**：大写的 `-A` 在标准的 `umount` 命令中没有这个功能。
-  **-t**
 - **理由**：用于指定 **ファイルシステムのタイプ (文件系统类型)**。例如 `umount -at nfs` 表示只卸载所有 NFS 类型的分区。它不能单独实现“全部卸载”。
-  **-ALL / -all**
 - **理由**：这些都不是标准的单字母或短格式选项。Linux 命令通常使用单字母（如 `-a`）或带有双横杠的长格式。

 `mount` 与 `umount` 的 `-a` 选项对比

命令	参照文件	行为 (日本語)
mount -a	/etc/fstab	挂载配置文件中列出的所有设备
umount -a	/etc/mtab	卸载当前已挂载的所有设备

🧠 小学生级记忆法：“全员放学 (All)”

想象 `mtab` 是教室里的「点名册」：

1. `-a` 代表 All (全部)。
2. 执行 `umount -a` 就像老师大喊一声：“全体 (All) 同学，放学回家！”
3. 于是，点名册 (`mtab`) 上记录的所有人都会离开教室（卸载）。

记住：不管是挂载还是卸载，“全都要”就找小写的 `-a`。

✅ 正确答案

`-a`

📝 考试小秘籍

- **拼写注意：**再次强调，命令是 `umount`，没有字母 `n`。
- **排除项：**执行 `umount -a` 时，系统通常很聪明，不会卸载正在运行的根目录 `/` 或内存虚拟文件系统，否则系统会立即崩溃。
- **搭配使用：**`-a` 经常和 `-t` 配合，比如 `umount -a -t cifs`（卸载所有 Windows 共享挂载）。

这一知识点主要考察 `umount` コマンド 的参数规则，即在卸载已挂载的文件系统时，如何正确指定目标。

💡 知识点总结

- `umount`：用于解除设备（如 `/dev/sdb2`）与挂载点（如 `/data`）之间的关联。
- **参数灵活性：**在卸载时，你可以通过指定「デバイス名 (设备名)」或「マウントポイント (挂载点)」中的 任意一个 来完成操作。
- **多目标支持：**`umount` 允许在同一行命令中列出多个参数，它会按顺序依次尝试卸载。

选项解析与详细说明

前提： /dev/sdb2 已经挂载到了 /data 。

- ☒ `umount /dev/sdb2`：正确答案。
 - 理由：通过 设备路径 进行卸载。这是最标准的做法之一。
- ☒ `umount -at xfs`
 - 理由：虽然命令格式合法，但它的意思是“卸载所有类型为 xfs 的设备”。这会把系统中所有其他的 xfs 分区也卸载掉，这通常不是题目要求的“针对性卸载”，且容易导致意外错误。
- ☒ `umount /data /dev/sdb2`：正确答案。
 - 理由：`umount` 支持同时指定多个参数。在这个例子中，它会先尝试卸载 /data （成功），然后尝试卸载 /dev/sdb2 。虽然第二次尝试会因为已经卸载而提示“not mounted”，但这条命令本身是 语法正确 且能完成目标的。
- ☒ `umount /data`：正确答案。
 - 理由：通过 挂载点路径 进行卸载。这是最常用、最方便的做法。
- ☒ `umount /dev/sdb2 /data`
 - 理由：逻辑同第三项。它会先通过设备名卸载成功，但在尝试卸载 /data 时，由于该目录已不再是挂载点，系统会报错。虽然最终结果是卸载了，但作为“正确且无错误”的选项，它不如其他选项严效。

 `umount` 卸载方式对比表

指定方式	命令示例	结果	备注
按设备名	<code>umount /dev/sdb2</code>	成功	常用方式
按挂载点	<code>umount /data</code>	成功	推荐方式
混合多个	<code>umount /data /dev/sdb1</code>	成功	依次执行卸载
按类型	<code>umount -a -t nfs</code>	成功	批量卸载特定类型

 小学生级记忆法：“分手协议”

想象 /dev/sdb2 （男生）和 /data （女生）正在谈恋爱（挂载状态）：

1. 单独叫人：你对男生说“你走吧”（`umount /dev/sdb2`），或者对女生说“你走吧”（`umount /data`），两人都会分手（卸载）。

2. **两个都叫**：你喊“男生走，女生也走”（`umount /data /dev/sdb2`）。第一位走了，第二位发现人已经没了，虽然有点尴尬，但分手（卸载）的目标确实达成了。
 3. **结论**：只要提名字，不管提谁的，都能拆散他们。
-

✅ 正确答案

- `umount /dev/sdb2`
 - `umount /data /dev/sdb2`
 - `umount /data`
-

📝 考试小秘籍

- **拼写红灯**：绝对不要写成 `unmount`（多了一个 n）。LPI 极其喜欢在这个字母上挖坑。
 - **Device Busy**：如果卸载报错“device is busy”，说明有程序正在 `/data` 目录下操作（比如你的终端正停在这个目录里）。你需要先退出该目录，或者用 `fuser -m /data` 查出是谁在占用。
 - **延迟卸载**：如果设备一直忙，可以使用 `umount -l` (lazy umount)，它会等设备不忙时自动完成卸载。
-

这三道题目共同考察了 Linux 中最强大的流编辑器 **sed (Stream Editor)** 的基本用法。它主要用于对文本进行 **置換 (替换)**、**削除 (删除)** 以及 **スクリプト (脚本)** 处理。

1. 多重编辑操作（置換 + 删除）

💡 知识点总结

- **-e オプション**：当需要在一个 `sed` 命令中执行 **多个编辑指令** 时，每个指令前都必须加上 `-e`。
- `s/old/new/g`：全行置換指令（`s` 代表 substitute，`g` 代表 global）。
- `/pattern/d`：匹配并删除指令（`d` 代表 delete）。

🔍 选项解析与详细说明

- ❌ `sed s/pingt/hoge/g/^#/d test.txt`：指令混在一起，语法错误。
- ❌ `sed -e s/pingt/hoge/g,/^#/d test.txt`：指令间不应该用逗号连接。
- ✅ `sed -e s/pingt/hoge/g -e /^#/d test.txt`：正确答案。

- **理由：**使用了两个 `-e` 正确连接了“替换”和“删除行头为 # 的行”这两个动作。
 -  `sed -e s/pingt/hoge/g /^#/d test.txt`：第二个指令缺少 `-e`。
-

2. 使用脚本文件进行编辑

知识点总结

- `-f オプション`：指定一个包含 `sed` 指令的 **ファイル (文件)**。`sed` 会读取该文件中的所有指令并作用于目标文本。

选项解析与详细说明

-  `sed -f editfile test.txt`：正确答案。
 - **理由：**`-f` 后直接跟文件名是标准用法。
 -  `sed -e editfile test.txt`：`-e` 后面应该直接跟指令字符串，而不是文件名。
 -  其他带 `/` 的选项：`/` 在 `sed` 中通常用于界定正则表达式，不用于包裹文件名。
-

3. 部分置换 vs 全局置换

知识点总结

- `s/old/new/` (无 `g`)：只替换每行中 **第一个 (最初)** 匹配到的字符串。
- `s/old/new/g` (带 `g`)：替换每行中 **所有 (全て)** 匹配到的字符串。
- **命令格式：**`sed` 命令可以带 `-e`，也可以在只有一个指令时省略 `-e`。

选项解析与详细说明

题目要求：仅置换“最初に現れる” (第一次出现) 的字符串。

-  `sed -e s/pingt/hoge/ test.txt`：正确答案。
 - **理由：**没有 `g` 标志，符合“仅第一次”的要求。
 -  `sed -e s/pingt/hoge/g test.txt`：带 `g` 会替换掉整行所有的匹配项。
 -  `grep ...`：`grep` 只能搜索，不能修改文本。
 -  `sed s/pingt/hoge/ test.txt`：正确答案。
 - **理由：**单个指令时，`-e` 可以省略，效果与第一项相同。
-



参数/标志	功能 (日本語)	用法说明
-e	編集スクリプトの指定	后面接指令字符串（多个指令需多次使用）
-f	スクリプトファイルの指定	后面接包含指令的文件名
s///	置換 (最初のみ)	只改每行的第一个
s///g	全置換 (全て)	改掉整行所有的匹配项
d	削除	删除匹配的行

🧠 小学生级记忆法：“sed 变身小精灵”

想象 `sed` 是一个拿画笔的小精灵：

1. `-e` (Execute)：精灵直接听你的口头指令：“把苹果变橘子！”
2. 多个 `-e`：精灵记性不好，你要说：“指令1：变橘子；指令2：擦掉香蕉！”
3. `-f` (File)：精灵拿出一张「任务清单 (File)」，照着上面的表格干活。
4. 最后没有 `g`：精灵很懒，每行只画一个就停手。
5. 加了 `g` (Global)：精灵充满活力，要把全世界（全行）的苹果都变掉！

✅ 正确答案总结

1. `sed -e s/pingt/hoge/g -e /^#/d test.txt`
2. `sed -f editfile test.txt`
3. `sed -e s/pingt/hoge/ test.txt` 及 `sed s/pingt/hoge/ test.txt`



考试小秘籍

- `-i` 选项：这是实战中最常用的。默认 `sed` 只是把结果显示在屏幕上，加了 `-i` (in-place) 才会真正修改原文件内容！
- `/` 的替代品：如果路径里有很多 `/`，你可以用 `s#old#new#g` 或者 `s|old|new|g`，效果是一样的。
- `p` 指令：通常配合 `-n` 使用，用于只显示特定的行（如 `sed -n '1,5p'` 显示 1 到 5 行）。

这几道题确实全部围绕 `grep` 系列命令 及其 オプション (选项) 展开。它们是 LPI 考试中关于文本过滤最核心的考点。我为你整理成了几个关键模块，方便你一并记忆。

💡 知识点总结

`grep` 命令家族主要分为三类，它们处理 正規表現 (正则表达式) 的方式不同：

- `grep`：基本正则表达式 (BRE)。
- `egrep` (等同于 `grep -E`)：扩展正则表达式 (ERE)，支持 `|` (或者)、`+`、`?` 等。
- `fgrep` (等同于 `grep -F`)：固定字符串搜索，**不识别**任何正则符号，把所有符号当普通文字处理。

🔍 选项解析与详细说明

3.1 搜索字面意思的 `.*` (特殊符号转义)

目标：寻找真正包含“点和星号”的行。

- ☒ `grep '\.*' test.txt`：在正则中 `.` 和 `*` 有特殊含义，必须加反斜杠 `\` 转义。
- ☒ `fgrep '.*' test.txt`：`fgrep` 不看正则，直接找 `.*` 这两个字符。
- ☒ `grep -F '.*' test.txt`：`-F` 与 `fgrep` 等价。

3.2 “A 或者 B” 的搜索 (OR 逻辑)

目标：查找 `PINGT` 或 `pingt`。

- ☒ `grep -E 'PINGT|pingt' test.txt`：使用 `-E` 开启扩展正则，支持 `|`。
- ☒ `egrep 'PINGT|pingt' test.txt`：`egrep` 原生支持 `|`。
- ☒ `egrep PINGT|pingt test.txt`：缺少引号。在 Linux 中 `|` 会被误认为管道符，导致命令报错。

3.3 排除特定行 (反向搜索)

目标：查找**不是**以 `#` 开头的行。

- ☒ `grep -v '^#' httpd.conf`：`^` 表示行头，`#` 是字符，`-v` 表示反转（排除）。

3.4 命令的基本格式与管道

- ☒ `grep PIN test.txt`：标准格式 `grep [模式] [文件]`。
- ☒ `cat test.txt | grep PIN`：通过管道符 `|` 将文本传递给 `grep` 处理。

grep 常用选项排查表

选项	功能 (日本語)	中文说明	记忆方法
-n	行番号をつけて表示	显示行号	Number
-c	マッチした行数のみ	只显示行数	Count
-v	マッチしなかった行	反向选择	Reverse
-i	大文字小文字を区別しない	忽略大小写	Ignore
-E	拡張正規表現を使用	扩展正则 (egrep)	Extended
-F	固定文字列として検索	固定字符串 (fgrep)	Fixed

小学生级记忆法：“grep 侦探事务所”

1. **-n (Number)**: 侦探不仅抓小偷，还顺便报告小偷住在“几号房”。
2. **-c (Count)**: 侦探不抓人，只拿个计数器数“有几个”小偷。
3. **-v (Invert)**: 侦探叛变了，他把好人全抓走，“留下”坏人。
4. **-i (Ignore)**: 侦探眼神不好，不管大写小写，只要长得像就抓。
5. **-F (Fixed)**: 侦探变呆了，他不再推理正则，只按照照片上的“死字符”找人。

正确答案总结

1. `grep '\.\\*' test.txt` / `fgrep '.*' test.txt` / `grep -F '.*' test.txt`
2. `grep -E 'PINGT|pingt' test.txt` / `egrep 'PINGT|pingt' test.txt`
3. `-n`
4. `-c`
5. `grep -v '^#' httpd.conf`
6. `cat test.txt | grep PIN` / `grep PIN test.txt`

考试小秘籍

- **引号的重要性**：凡是模式里包含 `|`，`*`，`.` 等特殊符号，**务必加单引号**，防止 Shell 提前解析它们。
 - **grep 的参数顺序**：记住是 `grep 模式 文件`，千万不要把文件名写在中间。
 - **fgrep 的速度**：在处理超大型日志文件且不需要正则时，`fgrep` 的性能是最快的。
-

这道题目考察的是 Linux 中「T字路口」般的命令：`tee コマンド`。它的作用是将数据流一分为二，既发送到 **標準出力 (屏幕)**，又保存到 **ファイル (文件)**。

💡 知识点总结

- **tee コマンド**：就像水管的 T 型接头，接收来自前一个命令的输出（通过管道 `|`），然后分^送到 **標準出力 (stdout)**：默认在屏幕上显示。
 - **ファイル保存**：将同样的内容写入指定的文件。
 - **重定向符号 >**：如果只用 `>`，内容会直接进文件，屏幕上什么也看不见。
-

🔍 选项解析与详细说明

目标：运行 `dmesg`（查看内核日志），同时看屏幕和存文件。

- ❌ `tee dmesg | log.txt`
 - **理由**：语法错误。`tee` 后面应该跟文件名，而不是命令名。
 - ❌ `tee log.txt < dmesg`
 - **理由**：`<` 是输入重定向，用于将文件内容传给命令，而 `dmesg` 是一个可执行程序，不是文件。
 - ✅ `dmesg | tee log.txt`：正确答案。
 - **理由**：
 - `dmesg` 产生输出。
 - `|` (管道) 将输出传给 `tee`。
 - `tee log.txt` 负责把这些内容显示在屏幕上，并同步写入 `log.txt`。
 - ❌ `dmesg > log.txt 2>&1`
 - **理由**：这会将标准输出和标准错误全都重定向到文件，屏幕上将**空无一物**。
 - ❌ `dmesg > log.txt`
 - **理由**：同上，这只是单纯的文件重定向，屏幕不显示。
-

重定向 vs tee 行为对比表

命令示例	屏幕显示	文件保存	备注
<code>dmesg > log.txt</code>	✗ (无)	✓ (保存)	覆盖原文件
<code>dmesg >> log.txt</code>	✗ (无)	✓ (保存)	追加到文件末尾
<code>**`dmesg</code>	<code>tee log.txt`**</code>	✓ (有)	✓ (保存)
<code>**`dmesg</code>	<code>tee -a log.txt`**</code>	✓ (有)	✓ (保存)

小学生级记忆法：“三通水管 (Tee)”

想象 `dmesg` 命令输出的是「自来水」：

1. 管道 `|`：是一根水管，把水引出来。
2. `tee` 命令：就是一个「T 型三通接头」。
3. 两个出口：一个口对着 屏幕（让你喝到水），另一个口接在 文件（把水存在水箱里）。
4. 长相：字母 `T` 的形状本身就代表了这种“一进二出”的分流结构！

口诀：屏幕文件都要看，中间必须加个 `tee` ！

正确答案

`dmesg | tee log.txt`

考试小秘籍

- 追加模式 `-a`：这是 `tee` 最常考的辅助选项。如果你不想覆盖旧文件，必须用 `tee -a` (append)。
- 错误重定向：如果你想把“错误信息”也通过 `tee` 分流，写法是 `dmesg 2>&1 | tee log.txt`。
- 管道顺序：`tee` 必须接在管道符 `|` 的后面，因为它处理的是“标准输入”。

这道题目考察的是 Linux 中最经典的「数据分流」工具。它的名字非常形象，直接对应了它的功能结构。

知识点总结

- `tee` コマンド：从 標準入力 (stdin) 读取数据，并将其内容同时输出到 標準出力 (stdout) 和 ファイル (文件)。
- 管道符 `|` 的搭档：通常紧跟在管道符后面，用于在处理数据的过程中“截留”一份副本存入文件。
- 追加模式：配合 `-a` 选项可以实现向文件末尾追加内容，而不是覆盖。

🔍 选项解析与详细说明

- ❌ `pea / pee / see / tea`
 - 理由：这些都是拼写干扰项。虽然 `pee` 在某些特定的管道工具包（如 `moreutils`）中存在，但在 LPI 考试和标准 Linux 认证中，这属于无效选项。
- ✅ `tee`：正确答案。
 - 理由：它的名字来源于大写字母「T」。就像水管的 T 型接头一样，进来的水流（输入）被分成了两个方向（显示器和文件）。

📊 数据流向对照表

操作方式	数据去向	屏幕显示	文件保存
<code>command > file</code>	仅文件	❌ 消失	✅ 覆盖
<code>**`command</code>	<code>tee file`**</code>	屏幕 + 文件	✅ 显示
<code>**`command</code>	<code>tee -a file`**</code>	屏幕 + 文件	✅ 显示

🧠 小学生级记忆法：“三通管 T (Tee)”

想象你正在修理家里的水管：

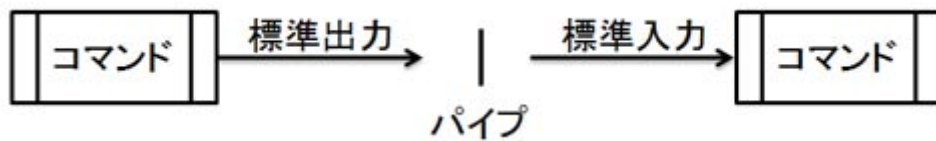
1. 你有一根主水管（标准输入）。
2. 你想让水流向洗手池（显示器），同时也想分出一部分水存进水桶（文件）。
3. 你需要安装一个「T 型接头」。
4. 这个接头的形状就是 `T`，所以命令就叫 `tee`。

口诀：屏幕文件两头走，中间全靠字母 T！

✅ 正确答案

✎ 考试小秘籍

- **-a 必考**：如果题目问“不删除原文件内容并输出”，一定要选 `tee -a`。
- **权限陷阱**：如果你想往一个只有 root 能写的系统文件里写东西，普通的 `sudo echo ... > file` 会失败，正确的做法是 `echo ... | sudo tee file`。
- **位置**：`tee` 必须放在管道 `|` 之后，因为它处理的是“输入流”。



这道题目考察的是 Linux 中最基础也最核心的符号：「パイプ (管道)」。它的作用是将一个命令的「標準出力 (输出)」直接连接到另一个命令的「標準入力 (输入)」。

💡 知识点总结

- `|` (パイプ)：用于连接两个命令，实现数据的流水线处理。
- `>`：出力リダイレクト (输出重定向)，将结果保存到 **文件**，而不是传给命令。
- `<`：入力リダイレクト (输入重定向)，将 **文件** 内容传给命令。
- `>>`：追加リダイレクト，将结果追加到文件末尾。
- `<<`：ヒアドキュメント (Here Document)，通常用于脚本中多行输入。

🔍 选项解析与详细说明

目标：把 `dmesg` 的显示结果交给 `less`（以便分页查看）。

- ❌ `dmesg > less`
 - **理由**：这会创建一个名为 `less` 的 **文件**，并把日志存进去。你不仅看不见内容，还会弄乱你的目录。
- ✅ `dmesg | less`：正确答案。
 - **理由**：
 - `dmesg` 输出大量的内核日志。
 - `|` (管道) 像一根管子一样接住这些日志。
 - `less` 接收这些日志，让你能用上下键滚动阅读。

- ❌ `dmesg << less`
 - 理由：语法错误。这是等待输入直到遇到“less”这个单词为止。
- ❌ `dmesg >> less`
 - 理由：将日志追加到一个名为 `less` 的文件中。
- ❌ `dmesg < less`
 - 理由：尝试把一个名为 `less` 的文件内容喂给 `dmesg` 命令，这逻辑完全反了。

 管道 vs 重定向对比表

符号	语法结构	右侧的对象	作用 (日本語)
<code>**`</code>	<code>`**</code>	<code>**命令</code>	命令 <code>**</code>
<code>></code>	命令 <code>></code> 文件	ファイル	ファイルへ保存

 小学生级记忆法：“接力棒 (Pipe)”

想象两个运动员在跑步：

- 运动员 A (`dmesg`)：手里拿着一堆情报（日志）。
- 运动员 B (`less`)：负责把情报整理好给大家看。
- 管道 `|`：就是他们之间交接的「接力棒」。
- 如果没有这根棒子，A 跑完了情报就扔地上了（屏幕刷屏而过），B 也没东西可以整理。

口诀：命令传给命令，中间立根棍子 (`|`)！

 正确答案

`dmesg | less`

 考试小秘籍

- 分清左右：管道左边是“产出者”，右边是“处理者”。
- 多重管道：你可以无限连接，如 `dmesg | grep eth0 | less`，这叫“链式处理”。
- less 的好处：在查看像 `dmesg` 这样几千行的输出时，直接回车会刷屏，用 `less` 才是运维老手的正确姿势。

以下のコマンドを実行した場合、表示される内容は次のうちどれか。「>」はプロンプトである。

```
$ cat << world
```

```
Hello world 1
```

```
Hello world 2
```

```
world
```

这一题考察的是 Linux Shell 中的「**ヒアドキュメント (Here Document)**」用法，这是一种在脚本中提供多行输入的特殊方式。

💡 知识点总结

- **<< (Here Document)**: 告诉 Shell，接下来的内容都是输入给前面命令（本题是 `cat`）的数据。
- **終了文字列 (结束标志)**: 紧跟在 `<<` 后面的字符串（本题是 `world`）定义了输入的 **边界**。
- **工作原理**: Shell 会持续接收输入，直到在一行中 **只看到** 这个结束标志为止。标志本身 **不属于** 内容的一部分，不会被显示。

🔍 选项解析与详细说明

我们来拆解一下命令的执行过程：

1. `cat << world` : 启动 `cat` 命令，并告诉它：“只要没看到单词 `world` 独占一行，你就一直收信。”
 2. `Hello world 1` : 第一行内容。
 3. `Hello world 2` : 第二行内容。
 4. `world` : 这一行匹配到了结束标志！于是输入停止。
 5. `cat` 的执行结果: 它会将接收到的两行内容打印出来。
- **✗ world**: 这是结束信号，不是内容。
 - **✗ Hello**: 内容不完整。
 - **✗ Hello world 2**: 只显示了第二行，不正确。
 - **✓ Hello world 1 \n Hello world 2**: 正确答案。
 - **理由**: `cat` 会原样输出结束标志之前的所有行。

📊 Here Document 结构拆解表

部分	内容	作用 (日本語)
cat << world	world	開始と終了の合図
中间行	Hello world 1 等	標準入力として扱われる内容
最后一行	world	入力の終わりを示す (表示されない)

🧠 小学生级记忆法：“大胃王挑战”

想象 `cat` 是一个正在参加「吃汉堡比赛」的大胃王：

1. `<< world`：比赛规则是：“只要我不说 `world` 这个词，你就一直给我塞汉堡！”
2. 第一个汉堡：`Hello world 1`（吃掉）。
3. 第二个汉堡：`Hello world 2`（吃掉）。
4. 大喊一声：`world`！（比赛结束）。
5. 结果：大胃王吐出来的（显示在屏幕上的）只有他刚才吃掉的那两个汉堡。

口诀：开头结尾要一致，中间才是真内容！

✅ 正确答案

Hello world 1

Hello world 2

📝 考试小秘籍

- 标志的选择：结束标志不一定要写 `world`，常用的还有 `EOF` (End Of File) 或 `STOP`。
- 引号的影响：如果写成 `<< "world"`（加引号），内容里的变量（如 `$HOME`）就不会被解析；如果不加引号，变量会被替换成真实的值。
- 缩进技巧：如果使用 `<<-`（多了一个减号），可以自动忽略掉内容行首的 Tab 缩进，这在写脚本时能让代码更好看。

这次我们重点讲解「重定向 (リダイレクト)」的高级用法，特别是如何把内容“接”在文件后面。

💡 知识点总结

- `>` (上書き/覆盖)：像橡皮擦，先擦干净文件里的所有内容，再写新的。（危险动作！）
- `>>` (追記/追加)：像盖楼，保留原来的内容，在 文件末尾 (末尾) 接着写新内容。

-  (入力/输入): 把文件内容“喂”给命令作为输入。

 选项解析与详细说明

目标: 将 `test1.txt` 的内容 **追加 (追記)** 到现有的 `tests.txt` 中。

-  `cat < test1.txt >> tests.txt` : 正确答案。
 - 理由: 虽然写得有点绕, 但逻辑是对的。先用 `<` 把 `test1.txt` 读进来, 再用 `>>` 追加到目的地。
-  `cat test1.txt | tests.txt`
 - 理由: 管道符 `|` 后面必须跟“命令”。`tests.txt` 是个文件, 系统会报错说“找不到这个命令”。
-  `cat test1.txt >> tests.txt` : 正确答案。
 - 理由: 这是 **最标准、最常用** 的写法。直接读取源文件并追加到目标文件。
-  `cat < test1.txt | tests.txt`
 - 理由: 同上, 管道后面不能直接接文件名。
-  `cat test1.txt > tests.txt`
 - 理由: 用了单个 `>`。这会 **删掉** `tests.txt` 里的旧内容, 不符合“追加”的要求。

 重定向符号快查表

符号	行为 (日本語)	对原文件内容的影响	记忆窍门
>	上書き	全部清除, 只留新的	只有一个头, 力量大, 会搞破坏
>>	追記	保留旧的, 末尾加新的	有两个头, 比较温柔, 往上堆叠
<	入力	无影响 (只读)	嘴巴张开, 吃掉文件

 小学生级记忆法: “搭积木 (积木遊び)”

1. `>` (覆盖): 看到一个新玩具, 就把旧的「全部扔掉」再放新的 (只有一根箭, 比较霸道)。
2. `>>` (追加): 看到新玩具, 就把它「放在旧的上面」继续搭高 (两根箭像梯子, 往上爬)。

口诀: 追回内容要两箭 (`>>`), 一箭穿心全擦掉 (`>`)!

 正确答案

- `cat < test1.txt >> tests.txt`
 - `cat test1.txt >> tests.txt`
-

考试小秘籍

- `2>` 与 `2>>`：这是考试的高频考点。`2` 代表 **标准错误 (stderr)**。如果你想把报错信息也存起来，记得用 `2>>`。
 - `&>`：如果你想把“正确的结果”和“报错的信息”通通存进同一个文件，可以用这个简写。
 - **文件不存在怎么办？**：无论是 `>` 还是 `>>`，如果目标文件不存在，系统都会 **自动创建一个新文件**。
-

这两道题考察的是 Linux 中最容易混淆的「**ファイルディスクリプタ (文件描述符)**」重定向。

在 Linux 中，每个打开的文件或流都有一个数字编号：

- `0`：標準入力 (stdin)
 - `1`：標準出力 (stdout) — 正常的结果。
 - `2`：標準エラー出力 (stderr) — 报错信息。
-

知识点总结

- `n>&m`：将编号为 `n` 的输出流合并（重定向）到编号为 `m` 的输出流中。
 - **& 符号的作用**：在重定向符号后面加 `&` 是为了告诉系统，后面的数字是一个「**文件描述符 (流)**」，而不是一个叫 "1" 或 "2" 的普通文件名。
-

选项解析与详细说明

第一题：将「标准错误 (2)」合并到「标准输出 (1)」

目标：让报错信息和正常结果一起显示或一起存入文件。

-  `2>&1`：正确答案。
 - **理由**：`2`（错误）重定向到 `&1`（输出）。这是最常用的写法，比如 `command > log.txt 2>&1`。
-  `1>&2`：这是反向操作。
-  `2>`：这只是把错误存入文件，并没有合并到标准输出。
-  `2>>`：这是把错误“追加”到文件末尾。

第二题：将「标准输出 (1)」合并到「标准错误 (2)」

目标：把正常的结果也当成错误流来处理（较少见，但在调试时会用到）。

- ✓ 1>&2：正确答案。
 - 理由：1（输出）重定向到 &2（错误）。
- ✗ 2>&1：这是第一题的答案。
- ✗ <<：这是之前讲过的 Here Document，用于多行输入。

📊 重定向组合排查表

符号	含义 (日本語)	中文说明	常用场景
2>&1	エラーを標準出力へ	错误转输出	把所有信息都存进一个 log
1>&2	標準出力をエラーへ	输出转错误	强制所有信息进入错误流
&>	両方をファイルへ	全部存文件	2>&1 的现代简写方式
2> /dev/null	エラーを捨てる	丢弃错误	只看正确结果，不看报错

🧠 小学生级记忆法：“左边分给右边”

你可以把 > 想象成一个「分流阀门」：

- 看左边的数字：它是“谁”要搬家？
- 看右边的数字：它是要搬到“哪”去？
- 2>&1：2号房 (错误) 的人要搬去 1号房 (输出) 住。
- 1>&2：1号房 (输出) 的人要搬去 2号房 (错误) 住。

口诀：数字在前地址后，& 带着房号走！

✓ 正确答案

- 2>&1
- 1>&2

📝 考试小秘籍

- 顺序很重要：在命令中，通常先写文件重定向，再写合并。例如 `ls > file 2>&1` 是对的，但 `ls 2>&1 > file` 可能会导致错误依然显示在屏幕上。

- **&> 简写**：LPI 考试有时会考 `&> filename`，它等同于 `> filename 2>&1`。
- **黑洞设备**：如果你不想看到任何报错，记住这个神器的组合：`2> /dev/null`。

这一题考查的是如何同时重定向「標準出力 (stdout)」和「標準エラー出力 (stderr)」到同一个文件的标准写法。

💡 知识点总结

- `> (1>)`：重定向标准输出（正确的结果）。
- `2>`：重定向标准错误（报错信息）。
- `2>&1`：将标准错误（2）合并到标准输出（1）中。
- **执行顺序**：Shell 会从左到右解析。通常先指定输出文件，再合并错误流。

🔍 选项解析与详细说明

目标：把 `test.sh` 的所有输出（正确+错误）都存入 `out.log`。

- ❌ `sh test.sh 1> 2> out.log`
 - **理由**：语法错误。不能连续使用两个重定向符号指向同一个文件。
- ❌ `sh test.sh 2> out.log`
 - **理由**：这只保存了 **错误信息**，正常的输出仍然会显示在屏幕上。
- ❌ `sh test.sh 2>&1 out.log`
 - **理由**：顺序错误且缺少重定向符号。系统会以为 `2>&1` 是传给脚本的参数，而 `out.log` 也没被正确指向。
- ❌ `sh test.sh 2>> out.log`
 - **理由**：这只是以 **追加** 模式保存错误信息，不包含标准输出。
- ✅ `sh test.sh > out.log 2>&1`：正确答案。
 - **理由**：
 - i. `> out.log`：先把标准输出（1）指向文件。
 - ii. `2>&1`：再把标准错误（2）送到标准输出（1）所在的地方。
 - iii. 结果：两者都进了 `out.log`。

📊 常见组合排查表

命令需求	正确语法 (标准版)	简写版 (现代 Shell)
只存正确结果	sh test.sh > out.log	-
只存报错信息	sh test.sh 2> out.log	-
全都要 (覆盖)	sh test.sh > out.log 2>&1	sh test.sh &> out.log
全都要 (追加)	sh test.sh >> out.log 2>&1	sh test.sh &>> out.log

小学生级记忆法：“排队上车”

想象有一辆叫 `out.log` 的校车：

- 1. 第一步 (`> out.log`)：班长（标准输出 1）先上车坐好。
- 2. 第二步 (`2>&1`)：副班长（标准错误 2）看到班长在哪，他就跟着去哪（ `&1` 表示“跟着 1 号走”）。
- 3. 结果：两个人都乖乖待在车里了。

口诀：先给 1 号找位子，再让 2 号跟 1 走！

✅ 正确答案

sh test.sh > out.log 2>&1

📝 考试小秘籍

- `&>` 简写：LPI 考试有时会给出 `sh test.sh &> out.log` 这种写法，这也是正确的，效果一模一样，而且更简洁。
- 顺序陷阱：千万别写成 `2>&1 > out.log`。如果这样写，错误信息会先飞向当时的屏幕，然后只有正常输出进了文件。
- 追加模式：如果题目要求“不覆盖原文件”，记得把 `>` 换成 `>>`。

这一题考查的是如何「特定 (专门)」重定向 標準エラー出力 (stderr)，而不干扰标准输出。

💡 知识点总结

- `2>`：标准错误重定向符号。其中的 `2` 代表文件描述符 `stderr`。
- 只捕获错误：如果你只想记录报错信息（比如权限不足、文件不存在等），而不关心成功的操作，就必须使用 `2>`。
- 命令格式：标准的写法是 `コマンド 引数 2> 出力先ファイル`。

🔍 选项解析与详细说明

目标：仅将 `rm ./*` 执行时的 **エラーメッセージ (错误信息)** 保存到 `rm.log`。

- ❌ `rm.log 2> rm ./*`
 - 理由：格式颠倒。系统会把 `rm.log` 当成命令执行，这显然不对。
- ❌ `rm.log < rm ./*`
 - 理由：`<` 是输入重定向，且位置完全错误。
- ✅ `rm ./* 2> rm.log`：正确答案。
 - 理由：
 - i. 执行 `rm ./*` 命令。
 - ii. `2>` 捕捉所有的报错信息（stderr）。
 - iii. 将这些报错存入 `rm.log`。如果操作成功，文件将是空的。
- ❌ `rm ./* >> rm.log`
 - 理由：这会把 **标准输出 (stdout)** 追加到文件，而我们要的是错误信息。
- ❌ `rm ./* > rm.log`
 - 理由：这会把 **标准输出 (stdout)** 覆盖到文件。

📊 错误重定向行为对照表

需求	命令示例	结果 (日本語)
仅保存错误 (覆盖)	<code>rm ./* 2> rm.log</code>	エラーのみ記録、上書き
仅保存错误 (追加)	<code>rm ./* 2>> rm.log</code>	エラーのみ記録、追記
保存错误和输出	<code>rm ./* &> rm.log</code>	全て記録
丢弃错误	<code>rm ./* 2> /dev/null</code>	エラーを表示しない

🧠 小学生级记忆法：“老二告状 (stderr)”

在 Linux 的数字家庭里：

1. **老大 (1)**：标准输出。他总是报告好消息（成功了！）。
2. **老二 (2)**：标准错误。他专门负责告状（出错了！）。
3. **任务**：如果你只想听告状的内容，就必须指名道姓让“**老二 (2)**”去文件里罚站。

口诀：二号老爱出差错， 2> 专抓错误货！

✅ 正确答案

```
rm ./2>rm.log*
```

📝 考试小秘籍

- 数字与符号之间不能有空格：写 2 > 是错的，必须紧挨着写成 2>。
 - 测试方法：你可以故意删一个不存在的文件来测试，比如 `rm non_exist_file 2>err.log`，看看 `err.log` 是不是记录了报错。
 - `/dev/null` 的妙用：在写自动化脚本时，如果不希望屏幕弹出乱七八糟的报错，经常会用到 `2> /dev/null`。
-

这三道题目涵盖了 Linux 中「リダイレクト (重定向)」的所有基本符号，是 LPI 101 考试中关于 I/O 操作的必考内容。

💡 知识点总结

- 标准流的编号：0 (stdin), 1 (stdout), 2 (stderr)。
 - 输出方向：> 和 >> 用于将结果从命令发往文件。
 - 输入方向：< 和 << 用于将内容喂给命令。
 - 覆盖与追加：单符号 (>) 通常是覆盖，双符号 (>>, <<) 通常与追加或持续输入有关。
-

🔍 选项解析与详细说明

第一题：切换「标准错误 (2)」的输出目的地

目标：让报错信息进入文件。

- ✅ 2>：正确答案。
 - 理由：将 標準エラー出力 覆盖写入文件。
- ✅ 2>>：正确答案。
 - 理由：将 標準エラー出力 追加到文件末尾。
- ❌ >> / < / <<：这些分别针对标准输出或输入。

第二题：切换「标准输出 (1)」的输出目的地

目标：让正常结果进入文件。

-  **>：正确答案。**
 - **理由：**将 **標準出力** 覆盖写入文件。
-  **>>：正确答案。**
 - **理由：**将 **標準出力** 追加到文件末尾。
-  **2> / 2>>：这些专门针对错误流。**

第三题：持续输入直到「終了文字 (结束标志)」

目标：实现多行输入 (Here Document)。

-  **<<：正确答案。**
 - **理由：**这就是「**ヒアドキュメント**」符号。它会一直等待输入，直到你输入了指定的结束单词。
-  **<：这是普通的输入重定向，用于读取 单个文件。**

 リダイレクト符号全览表

符号	流编号	行为 (日本語)	常用场景
>	1 (默认)	上書き (覆盖输出)	保存命令结果
>>	1 (默认)	追記 (追加输出)	记录日志
2>	2	エラーを上書き	记录报错信息
2>>	2	エラーを追記	持续记录报错
<	0	ファイルから入力	把文件喂给命令
<<	0	終了文字まで入力	脚本中生成多行文本

 小学生级记忆法：“箭头指向哪，数据去向哪”

1. 左边是命令，右边是文件：
 -  (一根箭)：力气大，把旧家具全扔了换新的（覆盖）。
 -  (两根箭)：力气匀称，把新东西叠在旧东西上面（追加）。
2. 带数字 2 的：是 “二号老二”，专门管报错。
3. 向左的箭 ：

- `<`：嘴巴对着文件，吃掉一个现成的文件。
- `<<`：嘴巴一直张着，等着你不停地喂，直到你说“吃饱了”（结束字符）。

口诀：一箭覆盖两箭加，数字 2 字报错家，左箭吃进右箭出，双左输入到结束！

✅ 正确答案

1. `2> / 2>>`
2. `/>>`
3. `<<`

📝 考试小秘籍

- 拼写细节：数字 `2` 必须和 `>` 紧挨着，中间不能有空格。
- 标准输出的省略：`1>` 效果等于 `>`。考试中通常省略 `1`。
- Here Document 的结束符：结束符通常用 `EOF` 或 `STOP`，但其实任何不带空格的单词都可以。

这一题考察的是 `tr` (translate) コマンド 的基本用法及其对「標準入力 (stdin)」的依赖性。

💡 知识点总结

- `tr` コマンド：用于 置換 (替换) 或 削除 (删除) 字符。
- 不接受文件名参数：这是 `tr` 最特别的地方——它不能直接像 `grep file.txt` 这样运行。它必须通过 `パイプ (|)` 或 `リダイレクト (<)` 接收数据。
- 逐字符替换：`tr PINGT pingt` 的含义是将所有的 `P` 换成 `p`，`I` 换成 `i`，以此类推。

🔍 选项解析与详细说明

目标：将 `file.txt` 中的 `PINGT` 转换并 显示在屏幕上。

- ❌ `file.txt | tr PINGT pingt`
 - 理由：`file.txt` 是一个文件，不是命令。你不能直接把文件名放在管道左边。
- ✅ `cat file.txt | tr PINGT pingt`：正确答案。
 - 理由：先用 `cat` 读取文件内容并输出，再通过管道传给 `tr` 进行替换，最后结果默认显示在屏幕上。
- ❌ `cat file.txt | tr PINGT pingt > display`

- **理由：**虽然替换成功了，但 `> display` 会把结果保存到一个名为 `display` 的 文件中，屏幕上反而看不见结果了。
-  **tr PINGT pingt < file.txt**：正确答案。
 - **理由：**使用输入重定向 `<` 将文件内容喂给 `tr`。这是 `tr` 最标准、最高效的用法之一。
-  **tr PINGT pingt < file.txt > display**
 - **理由：**同理，这会将结果存入文件，而不是显示在屏幕（显示器）上。

tr 命令使用规范表

模式	语法示例	是否正确	备注
错误示范	tr PINGT pingt file.txt	 错误	tr 不认文件名参数
管道模式	`cat file.txt	tr A a`	 正确
重定向模式	tr A a < file.txt	 正确	最推荐 的标准用法

小学生级记忆法：“没牙的小精灵 (tr)”

想象 `tr` 是一个专门负责把“大写字母”变成“小写字母”的 **小精灵**：

1. **他没牙：**他不会自己去咬开文件（`file.txt`）吃数据。
2. **得喂他：**你要么用吸管把水吸过来给他（**管道** `|`），要么把食物直接塞进他嘴里（**重定向** `<`）。
3. **结果：**他吃完吐出来的东西，默认就掉在你的桌子（**屏幕**）上。如果你拿个盆子接住（`>`），桌子上就没东西了。

口诀： `tr` 精灵不认路，管道重定向来喂部！

正确答案

- `cat file.txt | tr PINGT pingt`
- `tr PINGT pingt < file.txt`

考试小秘籍

- **字符集表示法：**考试常考 `tr '[:upper:]' '[:lower:]'`，这能一次性把所有大写转小写，比手动写 `A-Z` 更专业。

- **删除功能** `-d`：如果你想删掉所有空格，用 `tr -d ' '`。
- **压缩重复** `-s`：如果你想把连续的多个空格变成一个，用 `tr -s ' '`。

这一题考察的是 **vi (vim) エディタ** 的启动参数，特别是如何以「**読み取り専用 (只读)**」模式打开文件，以防止误操作修改重要配置文件。

 知识点总结

- **vi / vim**：Linux 中最常用的文本编辑器。
- **只读模式**：在该模式下，你可以查看文件内容，但如果你尝试保存 (`:w`)，系统会提示 “'readonly' option is set”，除非使用强制保存 (`:w!`)。
- **等价命令**：使用 `vi -R` 打开文件，其效果等同于直接使用 `view` 命令。

 选项解析与详细说明

目标：找到 vi 开启只读模式的正确选项。

-  **-ro / -RO**
 - **理由**：虽然看起来像 **Read-Only** 的缩写，但 vi 并不识别这两个选项。
-  **-read / -READ**
 - **理由**：这些都不是 vi 的标准短选项（短选项通常只有一位字母）。
-  **-R：正确答案。**
 - **理由**：大写的 `-R` 是 **Read-only** 的标准缩写。执行 `vi -R filename` 就能以只读方式打开文件。

 vi 启动常用选项对比表

选项	含义 (日本語)	用法说明
<code>-R</code>	読み取り専用	以只读模式打开，防止误删
<code>#NAME?</code>	指定した行から開く	打开文件并直接跳转到第 n 行
<code>-c</code>	コマンド実行	打开后立即执行某个 vi 命令

 小学生级记忆法：“大写的禁止 (R)”

想象你正在进入一个名为「**文件**」的房间：

1. 大写的 **R** (Read-only): 就像在门上贴了一张巨大的 **红色 (Red)** 警示牌。
2. 警告: 你可以进去 **看 (Read)**, 但 **大声 (大写)** 告诉自己: “**不准乱动东西!**”

口诀: 只读模式找大 R, 进门只能看风景!

✅ 正确答案

-R

📝 考试小秘籍

- **view 命令**: 考试经常会问“哪个命令等同于 `vi -R`”, 答案就是 `view`。
- **强制保存**: 即便用了 `-R` 模式, 如果你拥有该文件的写权限, 依然可以用 `:w!` 强行保存修改。
- **退出方式**: 只读模式下查看完, 直接输入 `:q` 即可退出。

コマンドモードから入力モードに移行する際に使用する主なviコマンドをまとめたものです。

コマンド	説明
i	カーソルの左から入力開始
a	カーソルの右から入力開始
I	カーソルのある行の先頭から入力開始
A	カーソルのある行の末尾から入力開始
o	カーソルのある行の下に空白行を挿入し、その行から入力開始
O	カーソルのある行の上に空白行を挿入し、その行から入力開始

这一题考察的是 **vi (vim) エディタ** 中从 **コマンドモード (命令模式)** 进入 **入力モード (插入模式)** 的不同快捷键。在 Linux 考试中, 这几个字母的细微区别是必考重点。

💡 知识点总结

- **i (insert)**: 在光标 **当前位置** 开始输入。
- **a (append)**: 在光标 **当前位置之后** 开始输入。
- **o (open line)**: 在光标 **当前行之下** 插入新行并开始输入。
- **O (Open line)**: 在光标 **当前行之上** 插入新行并开始输入。
- **A (Append to line)**: 在光标 **当前行的行尾** 开始输入。

选项解析与详细说明

目标：在光标「ある行の上 (所在行之上)」 插入空白行并输入。

-  **o**
 - **理由**：小写的 **o** 是在 **下方** 开启新行。记法：**open a line below**。
-  **O**：正确答案。
 - **理由**：大写的 **O** 是在 **上方** 开启新行。记法：**Open a line above**。
-  **i**
 - **理由**：就在光标那儿插，不会新开一行。
-  **A**
 - **理由**：飞到本行的最后面（行尾）去写东西。
-  **a**
 - **理由**：往后挪一个字符开始写。

vi 模式切换快捷键对比表

快捷键	动作 (日本語)	输入位置	记忆逻辑
i	挿入	光标处	insert (当前)
a	追加	光标后	append (后面)
o	下に1行挿入	下方新行	小 o 往下掉
O	上に1行挿入	上方新行	大 O 往上飘
A	行末に追加	行尾	All the way to end

小学生级记忆法：“大 O 向上，小 o 向下”

想象光标是一条地平线：

1. 小 **o**：像个小石头，受重力影响，**掉到下面** 砸出一个新坑（新行）。
2. 大 **O**：像个大氢气球，个头大有力气，**飘到上面** 顶出一个空间（新行）。

口诀：大 O 顶上，小 o 掉下！

正确答案

 考试小秘籍

- **大小写的威力**：vi 命令通常遵循“小写是常规，大写是增强/反向”的原则。比如 `o/O` 是上下，`p/P` 是粘贴到后/前。
- **回到命令模式**：无论你在哪个输入模式，按 `Esc` 键永远是你的救命稻草。
- **保存退出**：输入完后，记得输入 `:wq` 保存退出，或者 `:q!` 放弃修改退出。

odコマンドの書式と主なオプションは以下のとおりです。

od [オプション] [ファイル名]

オプション	説明
-t	出力フォーマットを指定
-o (デフォルト)	8進数2バイト区切りで表示 (-t o2と同じ)
-x	16進数2バイト区切りで表示 (-t x2と同じ)
-c	ASCII文字またはバックスラッシュ 付きのエスケープ文字で表示 (-t cと同じ)

这一题考察的是 `od (octal dump) コマンド` 的用法。虽然它的名字叫“八进制转储”，但它实际上是 Linux 中查看 `バイナリファイル (二进制文件)` 内容的通用工具。

 知识点总结

- `od コマンド`：将文件内容以八进制、十六进制、十进制或 ASCII 码形式显示。
- **デフォルト (默认行为)**：如果不加任何选项，`od` 默认以 **8進数 (八进制)** 显示。
- `-t オプション`：指定输出的 **タイプ (类型)**。
 - `x`：16進数 (Hexadecimal)
 - `o`：8進数 (Octal)
 - `d`：10進数 (Decimal)
 - `c`：ASCII 文字

 选项解析与详细说明

目标：以 **16進数 (十六进制)** 显示 `ls` 命令的二进制内容。

- ❌ `od /bin/ls`
 - 理由：默认会以 **8进数** 显示，不符合题目要求。
- ✅ `od -t x /bin/ls`：正确答案。
 - 理由：`-t x` 明确指定了类型为 **Hexadecimal** (16进数)。
- ❌ `head -c 16 /bin/ls`
 - 理由：这只会提取文件的前 16 个 **バイト (字节)**，且由于是二进制文件，直接输出到屏幕会乱码。
- ❌ `head -16 /bin/ls`
 - 理由：这会尝试提取文件的前 16 行，对二进制文件无效。
- ❌ `od -t o /bin/ls`
 - 理由：`-t o` 表示 **Octal** (8进数)。

 od 常用类型参数排查表

选项参数	显示格式 (日本語)	记忆窍门
-t x	16進数	Hex
-t o	8進数	Octal
-t d	10進数	Decimal
-t c	ASCII文字	Character

 小学生级记忆法：“翻译官 OD”

想象二进制文件是一本写满外星文（0 和 1）的书，`od` 是翻译官：

- 直接叫他翻译：他默认用最古老的 “8号方言” (八进制) 读给你听。
- `-t x`：你告诉他：“请用 ‘X 射线’ (Hex) 扫描一下，给我看十六进制的结果！”
- X 射线：射线扫过，原本看不见的 0101 变成了整齐的十六进制代码。

口诀：查看二进制找 `od`，十六进制加个 `x`！

✅ **正确答案**

`od -t x /bin/ls`

 考试小秘籍

- **hexdump 命令**：在实际工作中，`hexdump -C` 可能更常用，但在 LPI 101 考试中，`od` 才是大纲要求的核心考点。
- **组合使用**：如果你想同时看到十六进制和 ASCII 字符，可以使用 `od -t x1c`。
- **读取字节数**：配合 `-N` 选项可以只读取前 N 个字节，防止大文件刷屏（如 `od -t x -N 100 /bin/ls`）。

man 命令的書式と主なオプションは以下のとおりです。

man [オプション] [セクション番号] キーワード

オプション	説明
-k	キーワードに一部一致するコマンドやファイルの man ページを一覧表示 (apropos コマンドと同じ)
-f	キーワードに完全一致するコマンドやファイルの man ページを一覧表示 (whatis コマンドと同じ)

这一题考察的是 **man ページ (帮助手册)** 的搜索工具。在 Linux 中，搜索帮助文档主要有两种方式：一种是“关键字模糊搜索”，另一种是“名字精确匹配”。

 知识点总结

- **whatis コマンド**：用于查找与指定关键字 **完全一致 (完全一致)** 的命令名称，并简要显示其功能描述。
- **man -f オプション**：等同于 `whatis`。它会从索引数据库中查找匹配的手册页。
- **apropos / man -k**：这两个是 **キーワード検索 (关键字搜索)**，只要手册的描述中包含该词就会列出，结果非常多，不属于“完全一致”。

 选项解析与详细说明

目标：搜索与 `passwd` **完全一致** 的手册页。

-  **find -k passwd**
 - **理由**：`find` 是用来在文件系统中找文件的，`-k` 不是它的标准选项。
-  **apropos passwd**
 - **理由**：这会搜索所有包含 `passwd` 的描述，结果包括 `chpasswd`，`gpasswd` 等，不是“完全一致”。
-  **whatis passwd**：正确答案。
 - **理由**：`whatis` 专门用于查找命令名的 **完全匹配**。

-  `man -f passwd`：正确答案。
 - 理由：在 `man` 命令中，`-f` 选项的作用与 `whatis` 完全相同。
-  `man -k passwd`
 - 理由：这等同于 `apropos`，属于模糊搜索。

 man 搜索命令对比表

命令	等价选项	搜索范围	结果精确度
whatis	man -f	コマンド名のみ	高 (完全一致)
apropos	man -k	名前と説明文全体	低 (部分一致)

 小学生级记忆法：“名字叫啥 (What is)”

想象你在参加一个点名游戏：

1. `whatis` (他是谁?)：你大喊“张三！”，只有名字真叫“张三”的人才会站出来。这就是 完全一致。
2. `apropos` (关于...)：你大喊“关于张三！”，结果张三、张三丰、张三妮全都跑过来了。这就是 模糊搜索。
3. `-f` vs `-k`：
 - `-f` = Full match (全匹配，虽然官方叫法不是这个，但方便记忆)。
 - `-k` = Keyword (关键词，范围广)。

口诀：名字一致用 `whatis`，快捷方式 `man -f`！

 正确答案

- `whatis passwd`
- `man -f passwd`

 考试小秘籍

- 数据库更新：如果刚装了软件搜不到手册，记得先运行 `mandb` (或旧系统的 `makewhatis`) 来更新搜索数据库。

- **章节搜索：** `man` 手册分为不同章节（如 1 是普通命令，5 是配置文件）。你可以用 `man 5 passwd` 专门看配置文件的帮助。
- **返回值：**如果在考试中问你“哪个命令返回最简洁的结果”，通常也是 `whatis`。

这一题考察的是 Linux Shell 中的「ダブルクォート (双引号)」与「エスケープ (转义字符)」的组合用法。

💡 知识点总结

- **DATE= date``**：这是 **コマンド置換 (命令替换)**。它执行 `date` 命令，并将结果（如 `2026年 3月 3日...`）赋值给变量 `DATE`。
- **" (双引号)**：在双引号内，变量（如 `$DATE`）会被 **展開 (解析)** 为它的实际值。
- **\ (反斜杠)**：这是 **エスケープ文字 (转义字符)**。它会让紧跟在后面的特殊字符失去特殊魔力，变成普通的文字。

🔍 选项解析与详细说明

我们来拆解这条指令：`echo \"$DATE\"`

1. **\"**：这里的反斜杠作用于双引号。原本双引号是用来“包裹字符串”的工具，加了反斜杠后，它就变成了一个普通的“打印在屏幕上的引号字符”。
 2. **\$DATE**：由于它没有被反斜杠转义，Shell 会正常解析它，把它替换成 `date` 命令执行时的具体日時 (日期时间)。
 3. **最后的 \"**：同理，这也会在屏幕上打印一个双引号。
- **✗ "date"**：它输出的是变量的值，而不是字符串 "date"。
 - **✗ \"date\"**：反斜杠在执行时会被消耗掉，不会显示在屏幕上。
 - **✗ \"\$DATE\"**：变量会被解析，不会直接原样输出。
 - **✓ "dateコマンドを実行した時の日時"：正确答案。**
 - **理由：**实际输出结果看起来应该是 `"Tue Mar 3 00:10:03 JST 2026"`（带有左右双引号的日期字符串）。
 - **✗ "\$DATE"**：这缺了最外层的双引号。

📊 引号与转义字符效果对比表

假设变量 `DATE` 的值是 `2026-03-03`：

命令	输出结果 (日本語)	核心原理
echo \$DATE	2026/3/3	正常解析变量
echo "\$DATE"	2026/3/3	双引号内解析变量
echo '\$DATE'	\$DATE	シングルクォート内不解析变量
echo \" \$DATE \"	"2026-03-03"	转义引号 + 解析变量
echo "\\ \$DATE "	\$DATE	转义 \$ 符号，变量失效

小学生级记忆法：“卸妆的反斜杠”

想象双引号是一个正在变魔术的「魔法师」：

1. 正常情况：魔法师（"）能把变量变成真实的时间。
2. 遇到反斜杠 \：反斜杠就像一盆冷水，泼在魔法师身上，魔法师瞬间“卸妆”变成了普通人（普通的字符 "），再也无法变魔术了。
3. 结果：你会在屏幕上看到两个被泼了水的、呆呆的引号，中间夹着变出来的日期。

口诀：反斜杠是一盆水，泼谁谁就变原形！

✅ 正确答案

"dateコマンドを実行した時の日時"

（即：带有双引号的日期时间字符串）

📝 考试小秘籍

- 单引号 vs 双引号：这是 LPI 101 的必考点！单引号 ' ' 是“最强封印”，里面所有的 \$，\，` 都会失效。
- 反引号 `：它等同于 \$()。虽然现在推荐用 \$()，但考试中经常出现反引号，一定要认得它是执行命令的意思。
- 反斜杠自身：如果你想在屏幕上打出一个反斜杠，你需要写两个：echo \\\。

这一题考察的是「ダブルクォート (双引号)」对变量的处理方式。在 Shell 中，双引号虽然能识别变量，但它不会自动执行变量里存储的命令。

💡 知识点总结

- 变数 (变量) 的引用：使用 \$DATE 来读取变量 DATE 中存储的内容。

- **" (双引号)**: 在双引号内, Shell 会进行 **变数展開 (变量解析)**, 即把 `$DATE` 替换成它存储的字符串 "date".
- **与反引号的差异**:
 - **"\$DATE"**: 仅解析为字符串。
 - **`\$DATE`**: 解析为字符串后, 还会将其作为命令 **实行 (执行)**。

 选项解析与详细说明

假设执行了 `DATE="date"` :

- **✗ 現在の日時**
 - **理由**: 只有在加了 **反引号** ``` 或使用 `$( )` 时, 才会执行 `date` 命令并显示当前时间。
- **✓ date: 正确答案。**
 - **理由**: `echo "$DATE"` 的过程是:
 - i. 看到双引号里的 `$DATE` 。
 - ii. 把 `$DATE` 替换成它的值 `"date"` 。
 - iii. `echo` 打印出这个字符串 `"date"` 。
- **✗ \$date**: 格式错误, 变量名是大写的。
- **✗ \$DATE**
 - **理由**: 如果是单引号 `' $DATE '`, 才会原样输出这个符号。双引号会解析它。
- **✗ DATE**: 这是变量的名字, 不是它的内容。

 引用符号行为对比表

假设 `DATE="date"` :

命令	行为 (日本語)	输出结果	核心逻辑
<code>echo \$DATE</code>	变量解析	<code>date</code>	正常读取
<code>echo "\$DATE"</code>	变量解析	<code>date</code>	双引号允许解析
<code>echo '\$DATE'</code>	無効化	<code>\$DATE</code>	单引号强行不解析
<code>echo ` \$DATE `</code>	コマンド実行	<code>2026年 3月...</code>	执行变量里的指令

 小学生级记忆法: “照镜子 (双引号)”

想象变量 `DATE` 是一个穿着 “**date**” 字样衣服的小人：

1. 双引号 `"`：是一面「普通的镜子」。你对着镜子看（执行 `echo`），镜子里清清楚楚显示出小人衣服上的字：“**date**”。
2. 反引号 ```：是一面「魔法阵」。小人一站上去，衣服上的字就变成了「真实的动作」，小人立刻变出一块表给你看时间。

口诀：双引号里看文字，反引号里看动作！

✅ 正确答案

`date`

📝 考试小秘籍

- 变量赋值：记得赋值时不能有空格（`DATE="date"`）。
- `${DATE}`：这也是引用变量的写法，在大括号后面紧跟其他字符时非常有用，效果和 `$DATE` 一样。
- 单引号 vs 双引号：这是 LPI 101 的 必考 Top 3。记住：单引号最死板（啥都不变），双引号懂变通（解析变量），反引号最勤快（执行命令）。

这一题考察的是「シェル変数 (Shell 变量)」与「環境変数 (环境变量)」之间的转换。在 Linux 中，普通的变量只在当前 Shell 有效，如果想让子进程（如执行脚本时）也能看到这个变量，就必须进行 **エクスポート (导出)**。

💡 知识点总结

- シェル変数：仅在当前终端窗口（当前进程）内有效。
- 環境変数：由当前 Shell 及其派生的所有 **子プロセス (子进程)** 共享。
- 昇格 (升级)：将 Shell 变量转为环境变量的过程称为“导出”。

🔍 选项解析与详细说明

目标：将已经存在的变量 `LPIC` 变为环境变量。

- ❌ `printenv LPIC`
 - 理由：这是 **確認 (确认)** 命令。它用来打印环境变量的值，但不能把变量变成环境变量。
- ✅ `declare -x LPIC`：正确答案。

- **理由：** `declare` 是用来声明变量属性的命令。 `-x` 选项（`export`）的作用就是将变量导出为环境变量。
- **✗ `echo $LPIC`**
 - **理由：** 这只是在屏幕上 **表示 (显示)** 变量的值。
- **✓ `export LPIC` ： 正确答案。**
 - **理由：** 这是 **最常用** 的标准命令。它告诉 Shell：“把这个变量发给所有的子进程。”
- **✗ `export=LPIC`**
 - **理由：** 语法错误。 `export` 是命令，后面应该直接跟变量名，不需要等号。

 变量操作常用命令排查表

命令	作用 (日本語)	是否创建环境变量
LPIC=value	変数の定義	いいえ (仅为 Shell 变量)
export LPIC	環境変数へ昇格	はい
declare -x LPIC	環境変数へ昇格	はい
env	環境変数の一覧表示	-
unset LPIC	変数の削除	-

 小学生级记忆法：“出境签证 (Export)”

想象你的 Shell 变量是一个「小游客」：

1. **LPIC=study**：小游客刚出生，只能待在自家院子里（当前 Shell）。
2. **export** (出口/导出)：就像给小游客办了一张「出境签证」。
3. **结果**：有了签证，小游客就可以去别的国家（子进程、脚本、新的程序）旅游，并在那里被认出来。

口诀：变量想出门， `export` 盖个印；或者 `declare` ，带个 `-x` 证！

✓ 正确答案

- `export LPIC`
- `declare -x LPIC`

考试小秘籍

- **一步到位**：你也可以直接写 `export LPIC=study_linux`，这样在定义变量的同时就完成了导出。
- **注意大小写**：虽然不是强制规定，但习惯上 **环境变量** 通常使用 **大文字 (大写)**（如 `PATH`，`HOME`），而 **シェル变量** 使用 **小文字 (小写)**。
- **查看区别**：`set` 命令可以看到所有变量（包括 Shell 变量和环境变量），而 `env` 或 `printenv` 只能看到环境变量。

这一题考察的是「**变数 (变量)**」的赋值规则。在 Linux Shell 中，定义变量或给变量赋值时，**等号左右两边的细节** 是最容易出错的地方。

知识点总结

- **代入 (赋值)**：使用 `变数名=值` 的形式。
- **禁止空格**：在等号 `=` 的左右两边 **绝对不能有空格**。
- **不加 `$` 符号**：在 **定义** 或 **赋值** 时，左侧的变量名前面不要加 `$`。只有在 **引用 (读取)** 变量内容时才加 `$`。
- **一并导出**：使用 `export` 可以在赋值的同时将其变为 **环境变量**。

选项解析与详细说明

目标：将变量 `LPIC` 的值设置为 `test`。

-  `LPIC=test`：正确答案。
 - **理由**：这是最基础的 **シェル变量** 赋值方式。
-  `export $LPIC=test`
 - **理由**：错误地在左侧使用了 `$`。这会导致 Shell 尝试去解析 `$LPIC` 当前的值，而不是对 `LPIC` 这个名字进行操作。
-  `export LPIC=test`：正确答案。
 - **理由**：这不仅完成了赋值，还直接将它升格为 **环境变量**。这是合法且常用的写法。
-  `$LPIC=test`
 - **理由**：同上，赋值时左侧严禁使用 `$` 符号。
-  `LPIC test`
 - **理由**：缺少了关键的等号 `=`。

变量操作的正误对比表

命令示例	是否正确	错误原因 (日本語)
<code>LPIC=test</code>	OK	标准赋值
<code>LPIC = test</code>	NG	スペース (空格) がある
<code>\$LPIC=test</code>	NG	左側に \$ がついている
<code>export LPIC=test</code>	OK	環境変数として定義

小学生级记忆法：“名字标签 (赋值)”

想象你有一堆「空盒子」：

1. 赋值 (`LPIC=test`)：你拿一张标签写上“LPIC”，贴在装有“test”的盒子正面。这时候你手拿标签，不需要用 `$`。
2. 引用 (`echo $LPIC`)：你想看看盒子里装了什么，你就指着那个盒子说：“那个 (`$`) 叫 LPIC 的盒子里是什么？”

口诀：贴标签时不带钱 (`$`)，看内容时才叫钱 (`$`)；等号两边要抓紧，中间空格全踢走！

正确答案

- `LPIC=test`
- `export LPIC=test`

考试小秘籍

- 引号的使用：如果值里面包含空格（如 `LPIC="study linux"`），必须用双引号或单引号括起来。
- readonly：如果你想让一个变量被定义后不能被修改，可以使用 `readonly LPIC=test`。
- 清除变量：想把这个变量删掉？用 `unset LPIC`。

这一题考察的是 Linux 中「メモリ内 (内存中)」运行的特殊文件系统。这种文件系统读写速度极快，但断电或重启后数据会「消失 (消失)」。

💡 知识点总结

- **tmpfs (Temporary File System)**: Linux 内核支持的一种基于内存的文件系统。它不占用硬盘空间，而是直接使用 **RAM** 和 **Swap**。
- **速度优势**: 由于直接在内存中操作，I/O 性能远高于 SSD 或 HDD。
- **动态大小**: tmpfs 仅占用实际存储内容所需的内存量，不会一开始就霸占全额。
- **常见挂载点**: 在现代 Linux 系统中， /tmp、 /run 和 /dev/shm 通常都是 tmpfs 类型。

🔍 选项解析与详细说明

目标：找到在内存上创建的临时文件系统。

- **✗ memfs**
 - **理由**: 听起来很像，但在标准 Linux 内核中并没有这个叫法（某些特定操作系统如 Solaris 有类似名称，但 LPI 考的是 Linux）。
- **✗ VFAT**
 - **理由**: 这是 Windows 旧式的磁盘文件系统（如 U 盘常用），是持久化存储在 **物理磁盘** 上的。
- **✗ ext4**
 - **理由**: 这是目前 Linux 最常用的 **標準ディスクファイルシステム (标准磁盘文件系统)**，数据会永久保存。
- **✗ tmpfs**
 - **理由**: 这是一个迷惑选项，Linux 中并没有这个拼写的文件系统。
- **✔ tmpfs: 正确答案。**
 - **理由**: 这是官方定义的、专门用于内存的临时文件系统名称。

📊 文件系统特性对比表

特性	tmpfs	ext4 / XFS
存储介质	RAM (メモリ)	HDD / SSD (ディスク)
持久性	一時的 (重启丢失)	永続的 (永久保存)
访问速度	極めて高速	高速 (受限于硬件)
典型用途	/tmp、/run、缓存	/(根目录)、用户数据

🧠 小学生级记忆法：“黑板与笔记本”

- 1. **ext4 (笔记本)**: 你把字写在纸上，合上书明天还在。
- 2. **tmpfs (黑板)**: 下课铃响（重启），老师拿着“抹布 (tmp)” 过来一擦，黑板就“粉碎 (fs)” 变干净了。
- 3. 拼写记忆: TeMPorary File System → **tmpfs**。

口诀：临时文件用 `tmpfs`，重启一过全消失！

✅ 正确答案

tmpfs

✎ 考试小秘籍

- **挂载命令**: 考试可能会考如何挂载它。命令示例: `mount -t tmpfs -o size=1G tmpfs /mnt/mytmp`。
- **容量限制**: 默认情况下，`tmpfs` 的最大容量通常是物理内存的一半，但你可以通过 `size` 参数手动调整。
- `/dev/shm`: 这是 Linux 提供的一个默认 `tmpfs` 共享内存设备，非常有名的面试题/考点。

这一题考察的是 Linux 中先进的「**Btrfs (B-tree File System)**」。它被称为“下一代文件系统”，旨在解决传统文件系统（如 ext4）在可扩展性和数据完整性方面的局限。

💡 知识点总结

- **Btrfs**: 由 Oracle 开发，采用 **Copy-on-Write (CoW)** 机制的文件系统。

- **スナップショット (快照):** 可以瞬间创建文件系统的状态备份，且只占用新增数据的空间。
- **マルチデバイス (多设备):** 原生支持类似 RAID 的功能，可以将多个物理硬盘合并为一个逻辑池。
- **データ整合性:** 通过校验和 (Checksum) 自动检测并修复静默数据损坏。

🔍 选项解析与详细说明

- **✗ inodeの数が制限される**
 - **理由:** Btrfs 使用动态分配 inode 的机制，几乎 **没有数量限制**。相比之下，ext4 在格式化时就固定了 inode 数量。
- **✗ ファイルの圧縮はサポートしていない**
 - **理由:** Btrfs **支持透明压缩 (zlib, lzo, zstd)**，可以节省磁盘空间并延长 SSD 寿命。
- **✗ ext3の後継である**
 - **理由:** **ext4** 才是 ext3 的直接后继者。Btrfs 是一个全新的架构。
- **✅ サブボリューム単位でのスナップショット機能がある: 正确答案。**
 - **理由:** Btrfs 可以将文件系统划分为多个 **サブボリューム (子卷)**，并对每个子卷独立进行秒级快照备份。
- **✅ マルチデバイスに対応している: 正确答案。**
 - **理由:** 它内置了管理多个磁盘的能力，无需额外的 LVM 或硬件 RAID 即可实现条带化 (RAID 0) 或镜像 (RAID 1) 。

📊 Btrfs vs ext4 特性对照表

特性	Btrfs	ext4
写入机制	Copy-on-Write (CoW)	Journaling (日志)
快照功能	标准支持 (秒级)	不直接支持 (需结合 LVM)
磁盘管理	内置多设备支持 (RAID)	需配合 LVM / mdadm
文件压缩	支持	不支持
最大容量	16 EiB	1 EiB

🧠 小学生级记忆法: “瑞士军刀 (Btrfs)”

- 1. **Btrfs (瑞士军刀)**: 它不仅能切菜（存文件），还自带剪刀（压缩）、放大镜（校验）和好几个备用刀片（多设备 RAID）。
- 2. **快照 (照相机)**: 它随身带着相机，咔嚓一下（スナップショット），刚才的状态就永远记住了，随时能找回来。
- 3. **CoW (写作业)**: 如果你想改作业，它不是擦了重写，而是拿一张 **新纸 (Copy)** 写好再贴上去，这样原来的作业永远不会被弄坏。

口诀：多盘合并快照强，压缩备份 Btrfs 王！

✔ 正确答案

- サブボリューム単位でのスナップショット機能がある
- マルチデバイスに対応している

✎ 考试小秘籍

- **CoW 的代价**: 虽然 CoW 很安全，但会导致频繁随机写的数据库或虚拟机镜像文件产生严重的磁盘碎片，通常建议对这类文件禁用 CoW。
- **常用命令**: 考试可能会涉及 `btrfs subvolume create` 或 `btrfs device add` 等基础管理命令。
- **数据修复**: 记住 `btrfs scrub` 是用来检查和修复数据一致性的重要工具。

Linuxで使用できる主なテキストエディタは以下のとおりです。

エディタ	説明
Emacs	テキスト編集、コーディング、デバッグ、他プログラムの操作など開発環境として便利な機能を備えた高機能エディタ ユーザ独自の拡張機能をカスタマイズでき、GUIもサポートしている
nano	GUIをサポートしていない軽量の端末エディタ 操作キーとそれに対応する機能が画面下部に表示されており、初心者でも使いやすい
Vim	viから派生した高機能なテキストエディタ 基本的な操作方法是viと同じで、GUIもサポートしている 現在はviに代わり、Linuxで標準的に使用されている

这一题考察的是 Linux 系统中用于指定「標準エディタ (标准编辑器)」的环境变量。许多命令（如 `crontab -e`、`visudo` 或 `git commit`）在需要用户输入文字时，都会自动调用这个变量所指定的程序。

🧠 小学生级记忆法：“指挥官 (EDITOR)”

想象你的 Linux 系统是一个「交响乐团」：

1. **乐手们**： `crontab`、 `git` 这些程序都是乐手，他们偶尔需要有人出来“领唱”（写字）。
2. **指挥官 (EDITOR)**：你站在指挥台上大喊一声：“**EDITOR 是谁？**”
3. **结果**：如果变量里写着 `vim`，那么 `vim` 就会跳出来工作。如果你没设这个变量，乐手们可能会一脸懵逼，或者随便拉一个默认的人（通常是 `vi`）出来。

口诀：默认编辑选 `EDITOR`，想换 `nano` 随便设！

✅ 正确答案

EDITOR

📝 考试小秘籍

- **临时更改**：如果你想临时用 `nano` 编辑计划任务，可以执行 `EDITOR=nano crontab -e`。
- **与 VISUAL 的区别**：早期的 `EDITOR` 针对的是简单的行编辑器，而 `VISUAL` 针对的是全屏编辑器。现代系统中两者几乎通用，但 `VISUAL` 的优先级通常更高。
- **查看当前值**：使用 `echo $EDITOR` 来检查你当前的默认编辑器是谁。